

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

Introduction

This report documents the steps I followed to complete the cloud database and API assignment using a PostgreSQL database hosted on Neon, DbGate Community, and a FastAPI application. The main goal of the assignment was to practice working with a relational database in a cloud environment and to connect it to a backend API that performs real database operations. The work focused on setting up the database, writing SQL queries, and building API endpoints that interact with the database correctly. Screenshots are included throughout the report to show the results and confirm that each task was completed successfully.

The assignment was divided into several steps. I started by creating a cloud PostgreSQL database and connecting to it using DbGate. After that, I created the database schema and inserted sample data using SQL files. Once the database was ready, I ran the FastAPI application locally and verified that it could connect to the database. I then implemented API endpoints to retrieve products using SQL joins, create orders with stock updates, and generate summary statistics using aggregation and grouping queries.

During the assignment, I faced a few challenges. One issue was making sure that database connections were correctly configured inside the Docker environment. Another challenge was handling transactions when creating orders, so that stock levels, orders, and order items were updated safely without leaving partial data. I also needed to carefully test SQL joins and aggregate queries to ensure that the API responses matched the results shown in DbGate.

Step 1 — Create Neon PostgreSQL + connect with DbGate

In the first step, I set up the database environment that was required for all later parts of the assignment. I started by creating a new PostgreSQL project using the Neon cloud database platform. During the setup, I selected PostgreSQL version 17 and chose the AWS US East (Ohio) region. After the project was created, I obtained the database connection string and credentials provided by Neon.

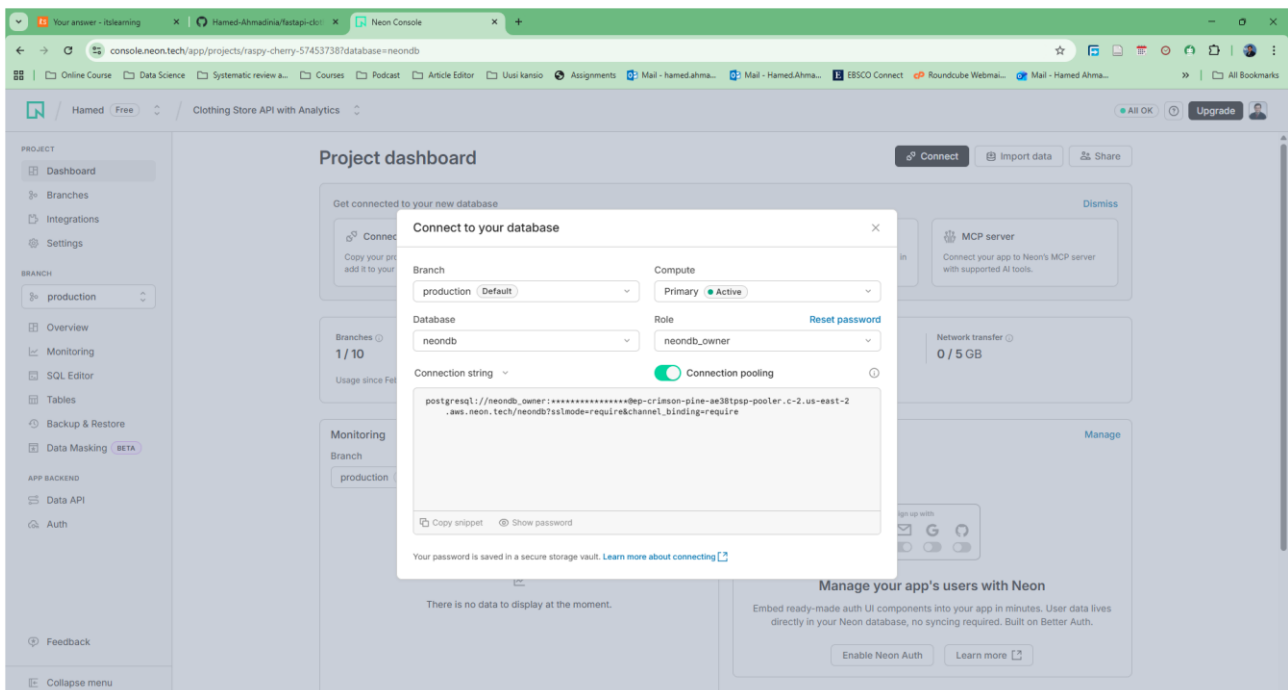
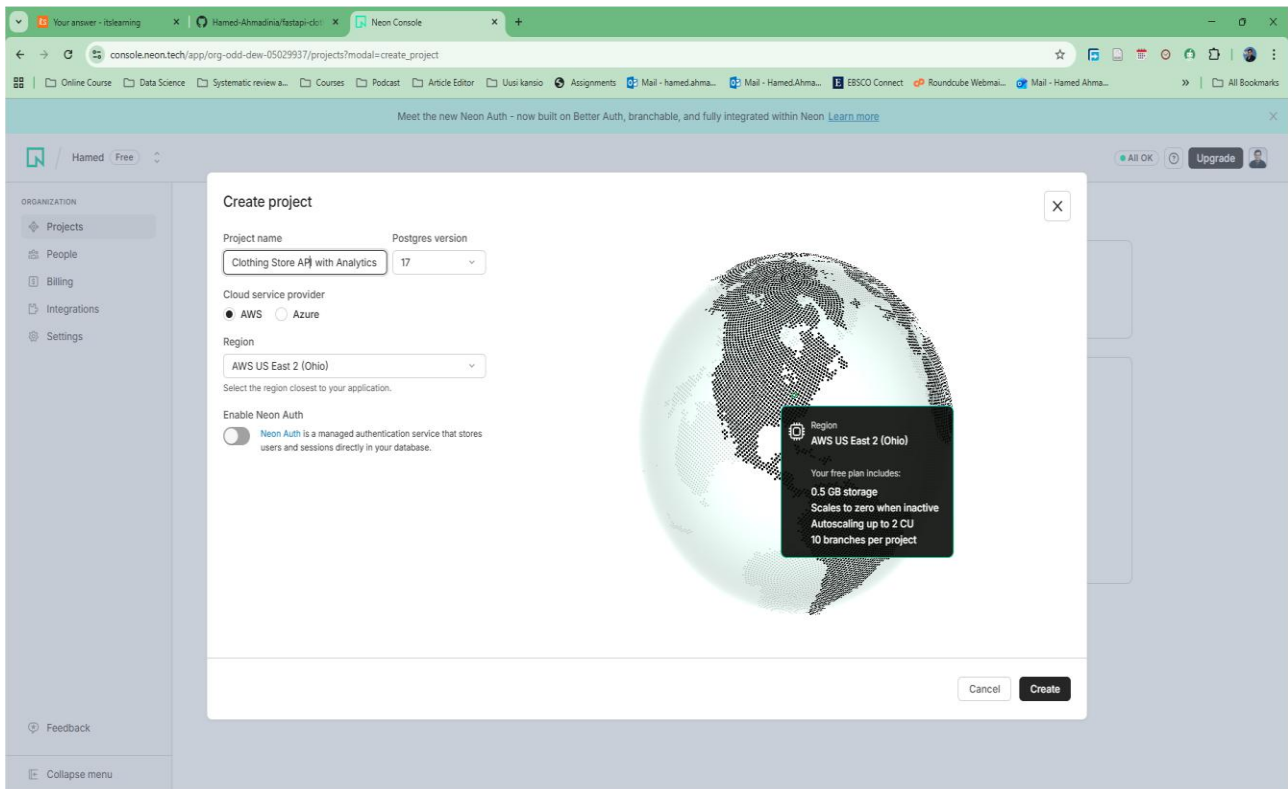
Next, I opened DbGate Community and created a new database connection. In the connection settings, I selected PostgreSQL as the database type and used the Neon connection string, username, and password. I tested the connection to ensure it was successful and then saved it.

Once connected, I verified the setup by running a simple SQL query to check the current database and user. I also confirmed that the database tables (such as categories, products, customers, orders, and order_items) were visible in the left panel. This confirmed that the database connection was working correctly and ready for executing SQL queries in the following steps. At this stage, the database connection was confirmed, and the environment was ready for schema creation in the next step.

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

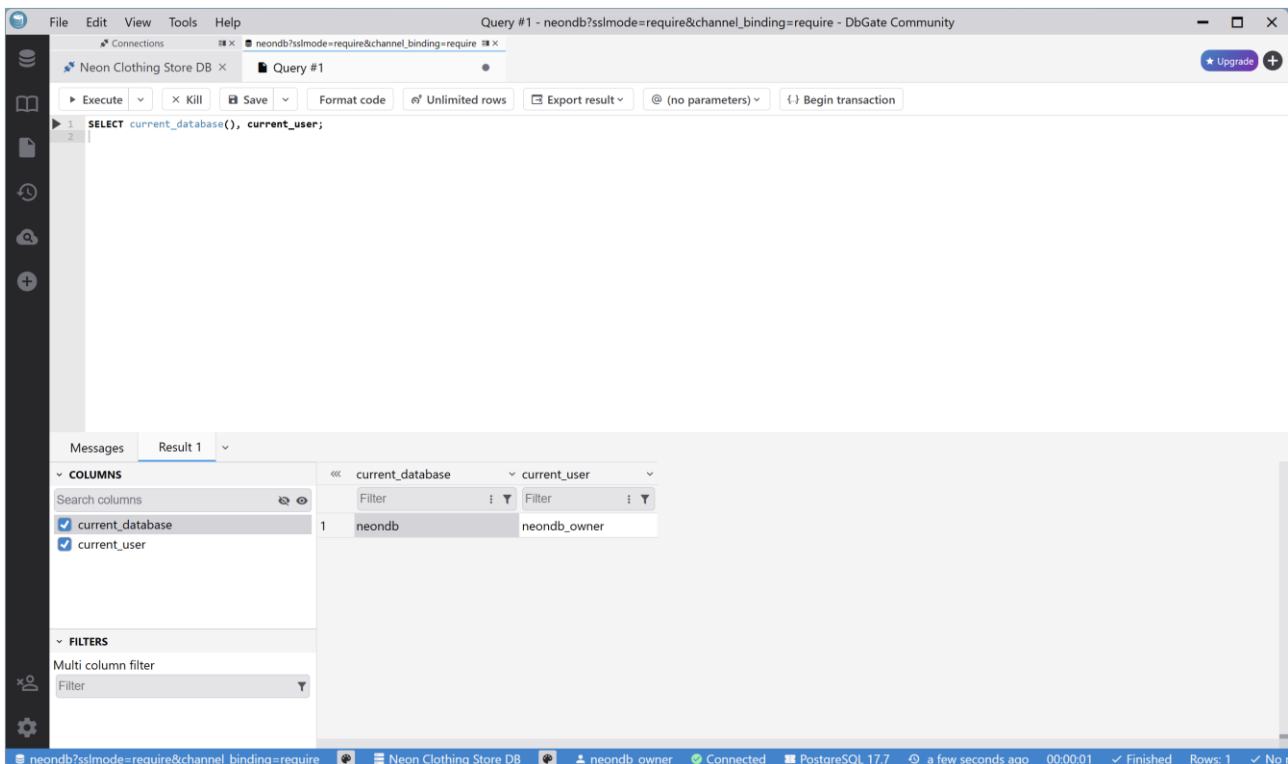
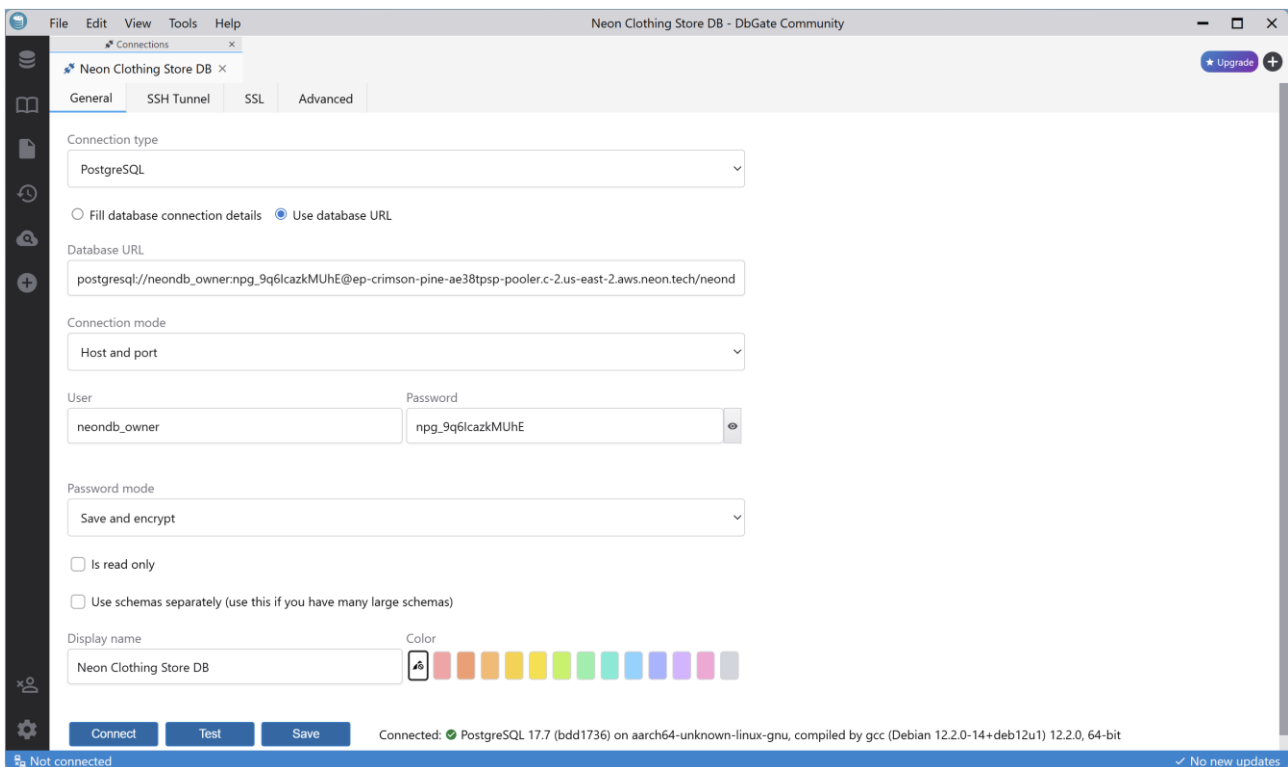
By: Hamed Ahmadinia



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

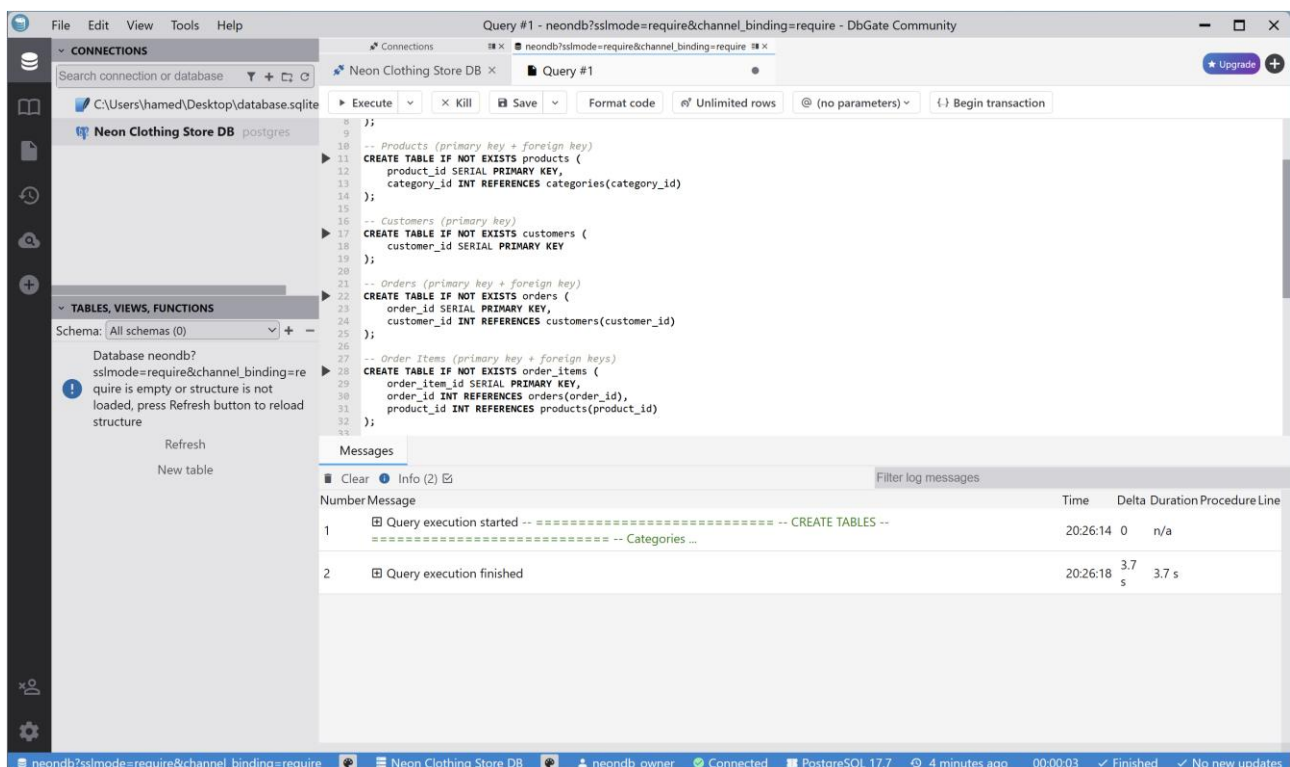
Step 2 — Seed schema + sample data (DbGate)

In the second step, I used DbGate Community to create the database schema and insert sample data needed for the application. After connecting to the Neon PostgreSQL database, I opened a new SQL query window in DbGate.

First, I opened the provided SQL files from the template repository and executed them in DbGate by copying and running them in order. Each table was created with a primary key, and foreign key relationships were added to link related tables correctly, such as products to categories and orders to customers. After running the queries, I confirmed that all tables were created successfully by checking that they appeared in the tables list on the left panel.

Next, I inserted sample data into the tables using INSERT statements. This included example categories, products, customers, orders, and order items. DbGate confirmed that the rows were inserted without errors and displayed the number of affected rows for each insert operation.

Finally, I verified the data by running a summary query that counted the number of rows in each table. The results showed that all tables contained data, confirming that the schema and sample data were correctly set up. This completed the database seeding step and prepared the database for use by the FastAPI application in later steps.



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface with a PostgreSQL connection named 'Neon Clothing Store DB'. The 'Query #1' tab is active, displaying a SQL script to create five tables: products, customers, orders, order_items, and categories. The 'Messages' pane at the bottom shows the execution progress, indicating that the query execution started at 20:26:14 and finished at 20:26:18, with a duration of 3.7 seconds.

```
-- Products (primary key + foreign key)
CREATE TABLE IF NOT EXISTS products (
  product_id SERIAL PRIMARY KEY,
  category_id INT REFERENCES categories(category_id)
);

-- Customers (primary key)
CREATE TABLE IF NOT EXISTS customers (
  customer_id SERIAL PRIMARY KEY
);

-- Orders (primary key + foreign key)
CREATE TABLE IF NOT EXISTS orders (
  order_id SERIAL PRIMARY KEY,
  customer_id INT REFERENCES customers(customer_id)
);

-- Order Items (primary key + foreign keys)
CREATE TABLE IF NOT EXISTS order_items (
  order_item_id SERIAL PRIMARY KEY,
  order_id INT REFERENCES orders(order_id),
  product_id INT REFERENCES products(product_id)
);
```

Number	Message	Time	Delta	Duration	Procedure	Line
1	Query execution started -- CREATE TABLES --	20:26:14	0	n/a		
2	Query execution finished	20:26:18	3.7 s	3.7 s		

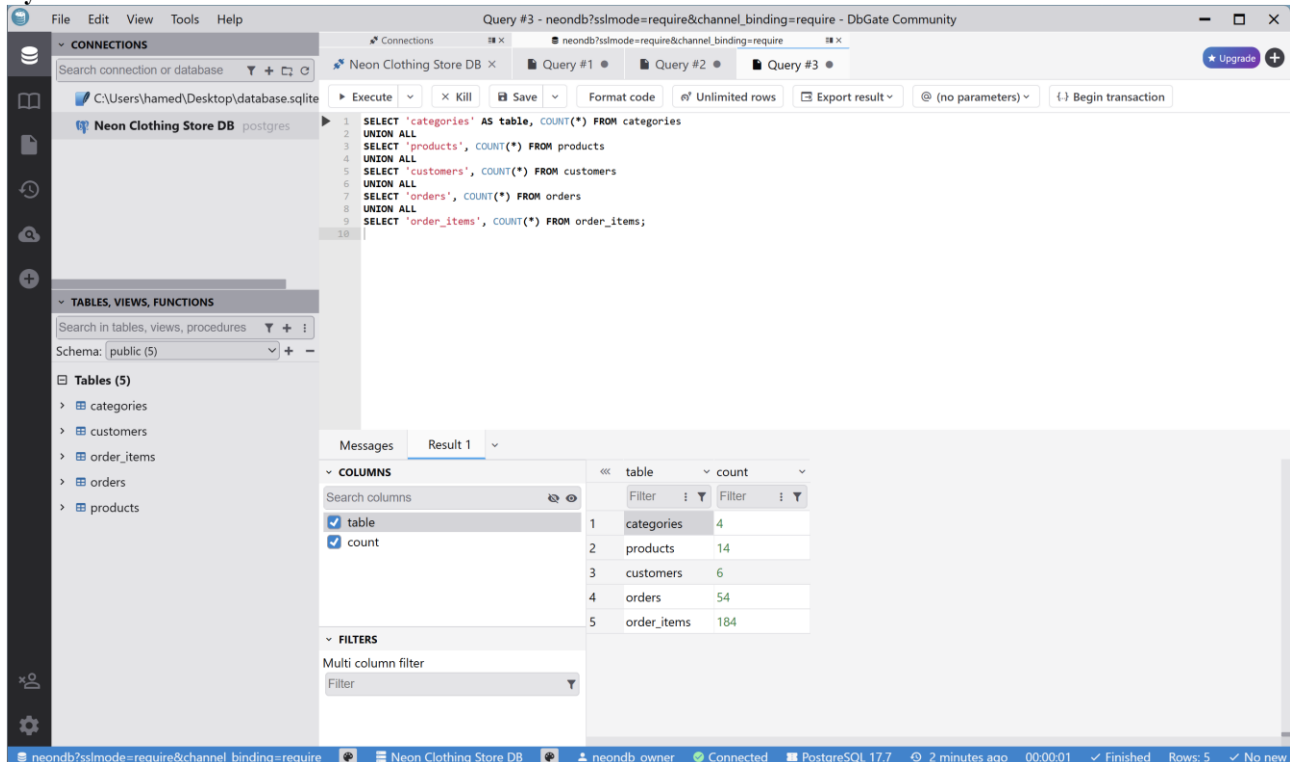
The screenshot shows the DbGate Community interface with the same PostgreSQL connection. The 'Query #2' tab is active, displaying a large SQL script to insert data into the tables. The 'Messages' pane at the bottom shows the execution progress, indicating that the query execution started at 20:27:47 and finished at 20:27:48, with a duration of 2.2 seconds.

```
(14,2,2),(14,7,1),(14,10,1),(14,12,1),
(15,3,1),(15,5,1),(15,8,2),(15,13,1),
(16,4,2),(16,6,1),(16,9,1),(16,14,1),
(17,1,1),(17,7,2),(17,10,1),(17,11,1),
(18,2,1),(18,5,2),(18,8,1),(18,12,1),
(19,3,2),(19,6,1),(19,9,1),(19,13,1),
(20,4,1),(20,7,1),(20,10,2),(20,14,1),
(21,1,2),(21,5,1),(21,8,1),(21,11,1),
(22,2,1),(22,5,2),(22,8,1),(22,12,1),
(23,3,1),(23,7,1),(23,10,1),(23,13,2),
(24,4,2),(24,5,1),(24,8,1),(24,14,1),
(25,1,1),(25,5,1),(25,8,2),(25,11,1),
(26,2,2),(26,7,1),(26,10,1),(26,12,1),
(27,3,1),(27,5,2),(27,8,1),(27,13,1),
(28,4,1),(28,6,1),(28,9,1),(28,14,2),
(29,1,2),(29,7,1),(29,10,1),(29,11,1),
(30,2,1),(30,5,1),(30,8,2),(30,12,1),
(31,3,2),(31,6,1),(31,9,1),(31,13,1),
(32,4,1),(32,7,2),(32,10,1),(32,14,1),
(33,1,1),(33,5,1),(33,8,1),(33,11,2),
(34,2,2),(34,6,1),(34,9,1),(34,12,1),
(35,3,1),(35,7,1),(35,10,1),(35,13,1),
(36,4,2),(36,5,1),(36,8,2),(36,14,1),
(37,1,1),(37,5,1),(37,8,2),(37,11,1),
(38,2,2),(38,7,1),(38,10,1),(38,12,1),
(39,3,1),(39,5,2),(39,8,1),(39,13,1),
(40,4,1),(40,6,1),(40,9,1),(40,14,2),
(41,1,2),(41,7,1),(41,10,1),(41,11,1),
(42,2,1),(42,5,1),(42,8,2),(42,12,1),
(43,3,2),(43,6,1),(43,9,1),(43,13,1),
(44,4,1),(44,7,1),(44,10,1),(44,14,2),
(45,1,2),(45,7,1),(45,10,1),(45,11,1),
(46,2,1),(46,5,1),(46,8,2),(46,12,1),
(47,3,2),(47,6,1),(47,9,1),(47,13,1),
(48,4,1),(48,7,1),(48,10,1),(48,14,2),
(49,1,2),(49,7,1),(49,10,1),(49,11,1),
(50,2,1),(50,5,1),(50,8,2),(50,12,1),
(51,3,2),(51,6,1),(51,9,1),(51,13,1),
(52,4,1),(52,7,1),(52,10,1),(52,14,2),
(53,1,2),(53,7,1),(53,10,1),(53,11,1),
(54,2,1),(54,5,1),(54,8,2),(54,12,1),
(55,3,2),(55,6,1),(55,9,1),(55,13,1),
(56,4,1),(56,7,1),(56,10,1),(56,14,2),
(57,1,2),(57,7,1),(57,10,1),(57,11,1),
(58,2,1),(58,5,1),(58,8,2),(58,12,1),
(59,3,2),(59,6,1),(59,9,1),(59,13,1),
(60,4,1),(60,7,1),(60,10,1),(60,14,2),
(61,1,2),(61,7,1),(61,10,1),(61,11,1),
(62,2,1),(62,5,1),(62,8,2),(62,12,1),
(63,3,2),(63,6,1),(63,9,1),(63,13,1),
(64,4,1),(64,7,1),(64,10,1),(64,14,2),
(65,1,2),(65,7,1),(65,10,1),(65,11,1),
(66,2,1),(66,5,1),(66,8,2),(66,12,1),
(67,3,2),(67,6,1),(67,9,1),(67,13,1),
(68,4,1),(68,7,1),(68,10,1),(68,14,2),
(69,1,2),(69,7,1),(69,10,1),(69,11,1),
(70,2,1),(70,5,1),(70,8,2),(70,12,1),
(71,3,2),(71,6,1),(71,9,1),(71,13,1),
(72,4,1),(72,7,1),(72,10,1),(72,14,2),
(73,1,2),(73,7,1),(73,10,1),(73,11,1),
(74,2,1),(74,5,1),(74,8,2),(74,12,1),
(75,3,2),(75,6,1),(75,9,1),(75,13,1),
(76,4,1),(76,7,1),(76,10,1),(76,14,2),
(77,1,2),(77,7,1),(77,10,1),(77,11,1),
(78,2,1),(78,5,1),(78,8,2),(78,12,1),
(79,3,2),(79,6,1),(79,9,1),(79,13,1),
(80,4,1),(80,7,1),(80,10,1),(80,14,2),
(81,1,2),(81,7,1),(81,10,1),(81,11,1),
(82,2,1),(82,5,1),(82,8,2),(82,12,1),
(83,3,2),(83,6,1),(83,9,1),(83,13,1),
(84,4,1),(84,7,1),(84,10,1),(84,14,2),
(85,1,2),(85,7,1),(85,10,1),(85,11,1),
(86,2,1),(86,5,1),(86,8,2),(86,12,1),
(87,3,2),(87,6,1),(87,9,1),(87,13,1),
(88,4,1),(88,7,1),(88,10,1),(88,14,2),
(89,1,2),(89,7,1),(89,10,1),(89,11,1),
(90,2,1),(90,5,1),(90,8,2),(90,12,1),
(91,3,2),(91,6,1),(91,9,1),(91,13,1),
(92,4,1),(92,7,1),(92,10,1),(92,14,2),
(93,1,2),(93,7,1),(93,10,1),(93,11,1),
(94,2,1),(94,5,1),(94,8,2),(94,12,1),
(95,3,2),(95,6,1),(95,9,1),(95,13,1),
(96,4,1),(96,7,1),(96,10,1),(96,14,2),
(97,1,2),(97,7,1),(97,10,1),(97,11,1),
(98,2,1),(98,5,1),(98,8,2),(98,12,1),
(99,3,2),(99,6,1),(99,9,1),(99,13,1),
(100,4,1),(100,7,1),(100,10,1),(100,14,2),
(101,1,2),(101,7,1),(101,10,1),(101,11,1),
(102,2,1),(102,5,1),(102,8,2),(102,12,1),
(103,3,2),(103,6,1),(103,9,1),(103,13,1),
(104,4,1),(104,7,1),(104,10,1),(104,14,2),
(105,1,2),(105,7,1),(105,10,1),(105,11,1),
(106,2,1),(106,5,1),(106,8,2),(106,12,1),
(107,3,2),(107,6,1),(107,9,1),(107,13,1),
(108,4,1),(108,7,1),(108,10,1),(108,14,2),
(109,1,2),(109,7,1),(109,10,1),(109,11,1),
(110,2,1),(110,5,1),(110,8,2),(110,12,1),
(111,3,2),(111,6,1),(111,9,1),(111,13,1),
(112,4,1),(112,7,1),(112,10,1),(112,14,2),
(113,1,2),(113,7,1),(113,10,1),(113,11,1),
(114,2,1),(114,5,1),(114,8,2),(114,12,1),
(115,3,2),(115,6,1),(115,9,1),(115,13,1),
(116,4,1),(116,7,1),(116,10,1),(116,14,2),
(117,1,2),(117,7,1),(117,10,1),(117,11,1),
(118,2,1),(118,5,1),(118,8,2),(118,12,1),
(119,3,2),(119,6,1),(119,9,1),(119,13,1),
(120,4,1),(120,7,1),(120,10,1),(120,14,2),
(121,1,2),(121,7,1),(121,10,1),(121,11,1),
(122,2,1),(122,5,1),(122,8,2),(122,12,1),
(123,3,2),(123,6,1),(123,9,1),(123,13,1),
(124,4,1),(124,7,1),(124,10,1),(124,14,2),
(125,1,2),(125,7,1),(125,10,1),(125,11,1),
(126,2,1),(126,5,1),(126,8,2),(126,12,1),
(127,3,2),(127,6,1),(127,9,1),(127,13,1),
(128,4,1),(128,7,1),(128,10,1),(128,14,2),
(129,1,2),(129,7,1),(129,10,1),(129,11,1),
(130,2,1),(130,5,1),(130,8,2),(130,12,1),
(131,3,2),(131,6,1),(131,9,1),(131,13,1),
(132,4,1),(132,7,1),(132,10,1),(132,14,2),
(133,1,2),(133,7,1),(133,10,1),(133,11,1),
(134,2,1),(134,5,1),(134,8,2),(134,12,1),
(135,3,2),(135,6,1),(135,9,1),(135,13,1),
(136,4,1),(136,7,1),(136,10,1),(136,14,2),
(137,1,2),(137,7,1),(137,10,1),(137,11,1),
(138,2,1),(138,5,1),(138,8,2),(138,12,1),
(139,3,2),(139,6,1),(139,9,1),(139,13,1),
(140,4,1),(140,7,1),(140,10,1),(140,14,2),
(141,1,2),(141,7,1),(141,10,1),(141,11,1),
(142,2,1),(142,5,1),(142,8,2),(142,12,1),
(143,3,2),(143,6,1),(143,9,1),(143,13,1),
(144,4,1),(144,7,1),(144,10,1),(144,14,2),
(145,1,2),(145,7,1),(145,10,1),(145,11,1),
(146,2,1),(146,5,1),(146,8,2),(146,12,1),
(147,3,2),(147,6,1),(147,9,1),(147,13,1),
(148,4,1),(148,7,1),(148,10,1),(148,14,2),
(149,1,2),(149,7,1),(149,10,1),(149,11,1),
(150,2,1),(150,5,1),(150,8,2),(150,12,1),
(151,3,2),(151,6,1),(151,9,1),(151,13,1),
(152,4,1),(152,7,1),(152,10,1),(152,14,2),
(153,1,2),(153,7,1),(153,10,1),(153,11,1),
(154,2,1),(154,5,1),(154,8,2),(154,12,1),
(155,3,2),(155,6,1),(155,9,1),(155,13,1),
(156,4,1),(156,7,1),(156,10,1),(156,14,2),
(157,1,2),(157,7,1),(157,10,1),(157,11,1),
(158,2,1),(158,5,1),(158,8,2),(158,12,1),
(159,3,2),(159,6,1),(159,9,1),(159,13,1),
(160,4,1),(160,7,1),(160,10,1),(160,14,2),
(161,1,2),(161,7,1),(161,10,1),(161,11,1),
(162,2,1),(162,5,1),(162,8,2),(162,12,1),
(163,3,2),(163,6,1),(163,9,1),(163,13,1),
(164,4,1),(164,7,1),(164,10,1),(164,14,2),
(165,1,2),(165,7,1),(165,10,1),(165,11,1),
(166,2,1),(166,5,1),(166,8,2),(166,12,1),
(167,3,2),(167,6,1),(167,9,1),(167,13,1),
(168,4,1),(168,7,1),(168,10,1),(168,14,2),
(169,1,2),(169,7,1),(169,10,1),(169,11,1),
(170,2,1),(170,5,1),(170,8,2),(170,12,1),
(171,3,2),(171,6,1),(171,9,1),(171,13,1),
(172,4,1),(172,7,1),(172,10,1),(172,14,2),
(173,1,2),(173,7,1),(173,10,1),(173,11,1),
(174,2,1),(174,5,1),(174,8,2),(174,12,1),
(175,3,2),(175,6,1),(175,9,1),(175,13,1),
(176,4,1),(176,7,1),(176,10,1),(176,14,2),
(177,1,2),(177,7,1),(177,10,1),(177,11,1),
(178,2,1),(178,5,1),(178,8,2),(178,12,1),
(179,3,2),(179,6,1),(179,9,1),(179,13,1),
(180,4,1),(180,7,1),(180,10,1),(180,14,2),
(181,1,2),(181,7,1),(181,10,1),(181,11,1),
(182,2,1),(182,5,1),(182,8,2),(182,12,1),
(183,3,2),(183,6,1),(183,9,1),(183,13,1),
(184,4,1),(184,7,1),(184,10,1),(184,14,2),
(185,1,2),(185,7,1),(185,10,1),(185,11,1),
(186,2,1),(186,5,1),(186,8,2),(186,12,1),
(187,3,2),(187,6,1),(187,9,1),(187,13,1),
(188,4,1),(188,7,1),(188,10,1),(188,14,2),
(189,1,2),(189,7,1),(189,10,1),(189,11,1),
(190,2,1),(190,5,1),(190,8,2),(190,12,1),
(191,3,2),(191,6,1),(191,9,1),(191,13,1),
(192,4,1),(192,7,1),(192,10,1),(192,14,2),
(193,1,2),(193,7,1),(193,10,1),(193,11,1),
(194,2,1),(194,5,1),(194,8,2),(194,12,1),
(195,3,2),(195,6,1),(195,9,1),(195,13,1),
(196,4,1),(196,7,1),(196,10,1),(196,14,2),
(197,1,2),(197,7,1),(197,10,1),(197,11,1),
(198,2,1),(198,5,1),(198,8,2),(198,12,1),
(199,3,2),(199,6,1),(199,9,1),(199,13,1),
(200,4,1),(200,7,1),(200,10,1),(200,14,2),
(201,1,2),(201,7,1),(201,10,1),(201,11,1),
(202,2,1),(202,5,1),(202,8,2),(202,12,1),
(203,3,2),(203,6,1),(203,9,1),(203,13,1),
(204,4,1),(204,7,1),(204,10,1),(204,14,2),
(205,1,2),(205,7,1),(205,10,1),(205,11,1),
(206,2,1),(206,5,1),(206,8,2),(206,12,1),
(207,3,2),(207,6,1),(207,9,1),(207,13,1),
(208,4,1),(208,7,1),(208,10,1),(208,14,2),
(209,1,2),(209,7,1),(209,10,1),(209,11,1),
(210,2,1),(210,5,1),(210,8,2),(210,12,1),
(211,3,2),(211,6,1),(211,9,1),(211,13,1),
(212,4,1),(212,7,1),(212,10,1),(212,14,2),
(213,1,2),(213,7,1),(213,10,1),(213,11,1),
(214,2,1),(214,5,1),(214,8,2),(214,12,1),
(215,3,2),(215,6,1),(215,9,1),(215,13,1),
(216,4,1),(216,7,1),(216,10,1),(216,14,2),
(217,1,2),(217,7,1),(217,10,1),(217,11,1),
(218,2,1),(218,5,1),(218,8,2),(218,12,1),
(219,3,2),(219,6,1),(219,9,1),(219,13,1),
(220,4,1),(220,7,1),(220,10,1),(220,14,2),
(221,1,2),(221,7,1),(221,10,1),(221,11,1),
(222,2,1),(222,5,1),(222,8,2),(222,12,1),
(223,3,2),(223,6,1),(223,9,1),(223,13,1),
(224,4,1),(224,7,1),(224,10,1),(224,14,2),
(225,1,2),(225,7,1),(225,10,1),(225,11,1),
(226,2,1),(226,5,1),(226,8,2),(226,12,1),
(227,3,2),(227,6,1),(227,9,1),(227,13,1),
(228,4,1),(228,7,1),(228,10,1),(228,14,2),
(229,1,2),(229,7,1),(229,10,1),(229,11,1),
(230,2,1),(230,5,1),(230,8,2),(230,12,1),
(231,3,2),(231,6,1),(231,9,1),(231,13,1),
(232,4,1),(232,7,1),(232,10,1),(232,14,2),
(233,1,2),(233,7,1),(233,10,1),(233,11,1),
(234,2,1),(234,5,1),(234,8,2),(234,12,1),
(235,3,2),(235,6,1),(235,9,1),(235,13,1),
(236,4,1),(236,7,1),(236,10,1),(236,14,2),
(237,1,2),(237,7,1),(237,10,1),(237,11,1),
(238,2,1),(238,5,1),(238,8,2),(238,12,1),
(239,3,2),(239,6,1),(239,9,1),(239,13,1),
(240,4,1),(240,7,1),(240,10,1),(240,14,2),
(241,1,2),(241,7,1),(241,10,1),(241,11,1),
(242,2,1),(242,5,1),(242,8,2),(242,12,1),
(243,3,2),(243,6,1),(243,9,1),(243,13,1),
(244,4,1),(244,7,1),(244,10,1),(244,14,2),
(245,1,2),(245,7,1),(245,10,1),(245,11,1),
(246,2,1),(246,5,1),(246,8,2),(246,12,1),
(247,3,2),(247,6,1),(247,9,1),(247,13,1),
(248,4,1),(248,7,1),(248,10,1),(248,14,2),
(249,1,2),(249,7,1),(249,10,1),(249,11,1),
(250,2,1),(250,5,1),(250,8,2),(250,12,1),
(251,3,2),(251,6,1),(251,9,1),(251,13,1),
(252,4,1),(252,7,1),(252,10,1),(252,14,2),
(253,1,2),(253,7,1),(253,10,1),(253,11,1),
(254,2,1),(254,5,1),(254,8,2),(254,12,1),
(255,3,2),(255,6,1),(255,9,1),(255,13,1),
(256,4,1),(256,7,1),(256,10,1),(256,14,2),
(257,1,2),(257,7,1),(257,10,1),(257,11,1),
(258,2,1),(258,5,1),(258,8,2),(258,12,1),
(259,3,2),(259,6,1),(259,9,1),(259,13,1),
(260,4,1),(260,7,1),(260,10,1),(260,14,2),
(261,1,2),(261,7,1),(261,10,1),(261,11,1),
(262,2,1),(262,5,1),(262,8,2),(262,12,1),
(263,3,2),(263,6,1),(263,9,1),(263,13,1),
(264,4,1),(264,7,1),(264,10,1),(264,14,2),
(265,1,2),(265,7,1),(265,10,1),(265,11,1),
(266,2,1),(266,5,1),(266,8,2),(266,12,1),
(267,3,2),(267,6,1),(267,9,1),(267,13,1),
(268,4,1),(268,7,1),(268,10,1),(268,14,2),
(269,1,2),(269,7,1),(269,10,1),(269,11,1),
(270,2,1),(270,5,1),(270,8,2),(270,12,1),
(271,3,2),(271,6,1),(271,9,1),(271,13,1),
(272,4,1),(272,7,1),(272,10,1),(272,14,2),
(273,1,2),(273,7,1),(273,10,1),(273,11,1),
(274,2,1),(274,5,1),(274,8,2),(274,12,1),
(275,3,2),(275,6,1),(275,9,1),(275,13,1),
(276,4,1),(276,7,1),(276,10,1),(276,14,2),
(277,1,2),(277,7,1),(277,10,1),(277,11,1),
(278,2,1),(278,5,1),(278,8,2),(278,12,1),
(279,3,2),(279,6,1),(279,9,1),(279,13,1),
(280,4,1),(280,7,1),(280,10,1),(280,14,2),
(281,1,2),(281,7,1),(281,10,1),(281,11,1),
(282,2,1),(282,5,1),(282,8,2),(282,12,1),
(283,3,2),(283,6,1),(283,9,1),(283,13,1),
(284,4,1),(284,7,1),(284,10,1),(284,14,2),
(285,1,2),(285,7,1),(285,10,1),(285,11,1),
(286,2,1),(286,5,1),(286,8,2),(286,12,1),
(287,3,2),(287,6,1),(287,9,1),(287,13,1),
(288,4,1),(288,7,1),(288,10,1),(288,14,2),
(289,1,2),(289,7,1),(289,10,1),(289,11,1),
(290,2,1),(290,5,1),(290,8,2),(290,12,1),
(291,3,2),(291,6,1),(291,9,1),(291,13,1),
(292,4,1),(292,7,1),(292,10,1),(292,14,2),
(293,1,2),(293,7,1),(293,10,1),(293,11,1),
(294,2,1),(294,5,1),(294,8,2),(294,12,1),
(295,3,2),(295,6,1),(295,9,1),(295,13,1),
(296,4,1),(296,7,1),(296,10,1),(296,14,2),
(297,1,2),(297,7,1),(297,10,1),(297,11,1),
(298,2,1),(298,5,1),(298,8,2),(298,12,1),
(299,3,2),(299,6,1),(299,9,1),(299,13,1),
(300,4,1),(300,7,1),(300,10,1),(300,14,2),
(301,1,2),(301,7,1),(301,10,1),(301,11,1),
(302,2,1),(302,5,1),(302,8,2),(302,12,1),
(303,3,2),(303,6,1),(303,9,1),(303,13,1),
(304,4,1),(304,7,1),(304,10,1),(304,14,2),
(305,1,2),(305,7,1),(305,10,1),(305,11,1),
(306,2,1),(306,5,1),(306,8,2),(306,12,1),
(307,3,2),(307,6,1),(307,9,1),(307,13,1),
(308,4,1),(308,7,1),(308,10,1),(308,14,2),
(309,1,2),(309,7,1),(309,10,1),(309,11,1),
(310,2,1),(310,5,1),(310,8,2),(310,12,1),
(311,3,2),(311,6,1),(311,9,1),(311,13,1),
(312,4,1),(312,7,1),(312,10,1),(312,14,2),
(313,1,2),(313,7,1),(313,10,1),(313,11,1),
(314,2,1),(314,5,1),(314,8,2),(314,12,1),
(315,3,2),(315,6,1),(315,9,1),(315,13,1),
(316,4,1),(316,7,1),(316,10,1),(316,14,2),
(317,1,2),(317,7,1),(317,10,1),(317,11,1),
(318,2,1),(318,5,1),(318,8,2),(318,12,1),
(319,3,2),(319,6,1),(319,9,1),(319,13,1),
(320,4,1),(320,7,1),(320,10,1),(320,14,2),
(321,1,2),(321,7,1),(321,10,1),(321,11,1),
(322,2,1),(322,5,1),(322,8,2),(322,12,1),
(323,3,2),(323,6,1),(323,9,1),(323,13,1),
(324,4,1),(324,7,1),(324,10,1),(324,14,2),
(325,1,2),(325,7,1),(325,10,1),(325,11,1),
(326,2,1),(326,5,1),(326,8,2),(326,12,1),
(327,3,2),(327,6,1),(327,9,1),(327,13,1),
(328,4,1),(328,7,1),(328,10,1),(328,14,2),
(329,1,2),(329,7,1),(329,10,1),(329,11,1),
(330,2,1),(330,5,1),(330,8,2),(330,12,1),
(331,3,2),(331,6,1),(331,9,1),(331,13,1),
(332,4,1),(332,7,1),(332,10,1),(332,14,2),
(333,1,2),(333,7,1),(333,10,1),(333,11,1),
(334,2,1),(334,5,1),(334,8,2),(334,12,1),
(335,3,2),(335,6,1),(335,9,1),(335,13,1),
(336,4,1),(336,7,1),(336,10,1),(336,14,2),
(337,1,2),(337,7,1),(337,10,1),(337,11,1),
(338,2,1),(338,5,1),(338,8,2),(338,12,1),
(339,3,2),(339,6,1),(339,9,1),(339,13,1),
(340,4,1),(340,7,1),(340,10,1),(340,14,2),
(341,1,2),(341,7,1),(341,10,1),(341,11,1),
(342,2,1),(342,5,1),(342,8,2),(342,12,1),
(343,3,2),(343,6,1),(343,9,1),(343,13,1),
(344,4,1),(344,7,1),(344,10,1),(344,14,2),
(345,1,2),(345,7,1),(345,10,1),(345,11,1),
(346,2,1),(346,5,1),(346,8,2),(346,12,1),
(347,3,2),(347,6,1),(347,9,1),(347,13,1),
(348,4,1),(348,7,1),(348,10,1),(348,14,2),
(349,1,2),(349,7,1),(349,10,1),(349,11,1),
(350,2,1),(350,5,1),(350,8,2),(350,12,1),
(351,3,2),(351,6,1),(351,9,1
```

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania



The screenshot shows the DbGate Community interface. The main window displays a SQL query for Query #3, which is a UNION ALL query counting records from five tables: categories, products, customers, orders, and order_items. The query is as follows:

```
1 SELECT 'categories' AS table, COUNT(*) FROM categories
2 UNION ALL
3 SELECT 'products', COUNT(*) FROM products
4 UNION ALL
5 SELECT 'customers', COUNT(*) FROM customers
6 UNION ALL
7 SELECT 'orders', COUNT(*) FROM orders
8 UNION ALL
9 SELECT 'order_items', COUNT(*) FROM order_items;
```

The results are shown in a table with two columns: 'table' and 'count'.

table	count
categories	4
products	14
customers	6
orders	54
order_items	184

The interface also shows a sidebar with connections and tables. The 'Neon Clothing Store DB' connection is selected, and the 'public' schema is shown with tables: categories, customers, order_items, orders, and products.

Step 3 — Local FastAPI runs + DB connectivity verified

In this step, I configured and verified the local FastAPI application and confirmed that it could successfully connect to the PostgreSQL database. I first created a .env file in the project root and added the required environment variables, including the application mode and the database connection URL provided by Neon.

After setting the environment variables, I started the FastAPI application using Docker Compose. The build process completed successfully, and the container started without errors. The terminal logs confirmed that Uvicorn was running and listening on port 8080, and the application startup completed normally.

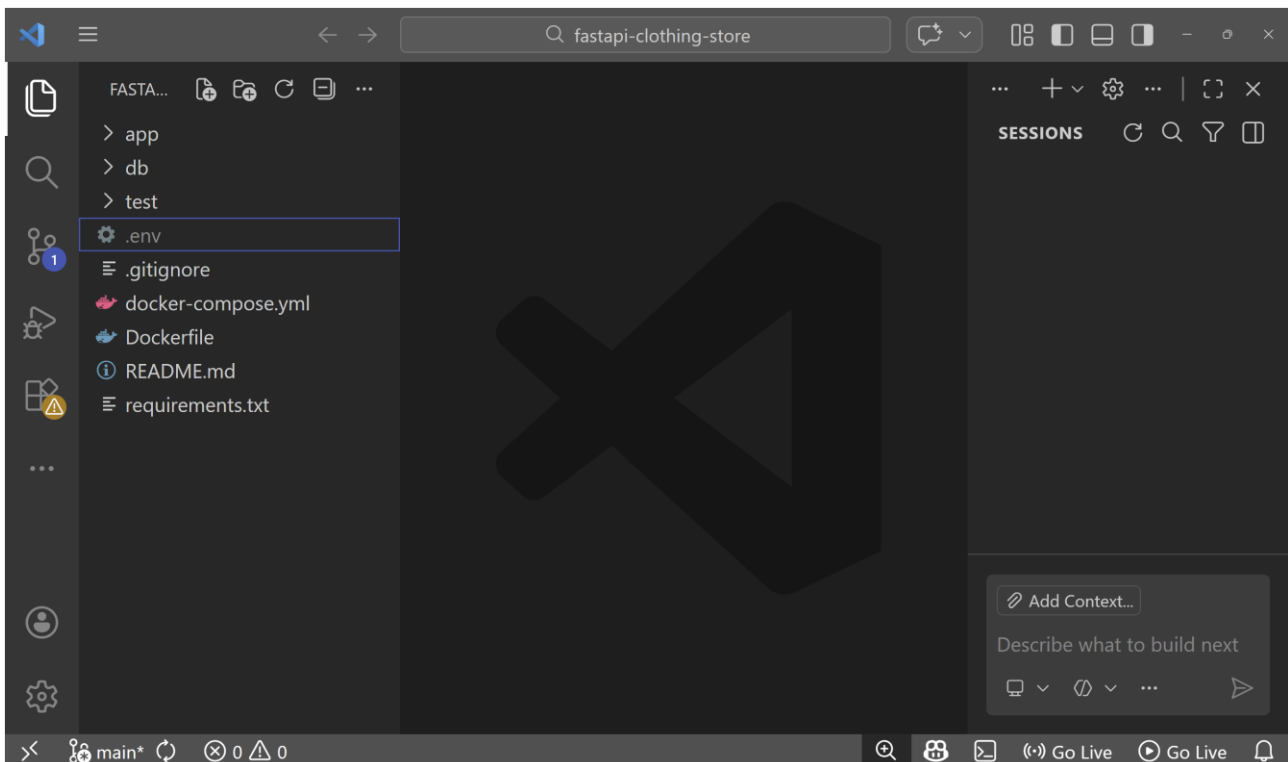
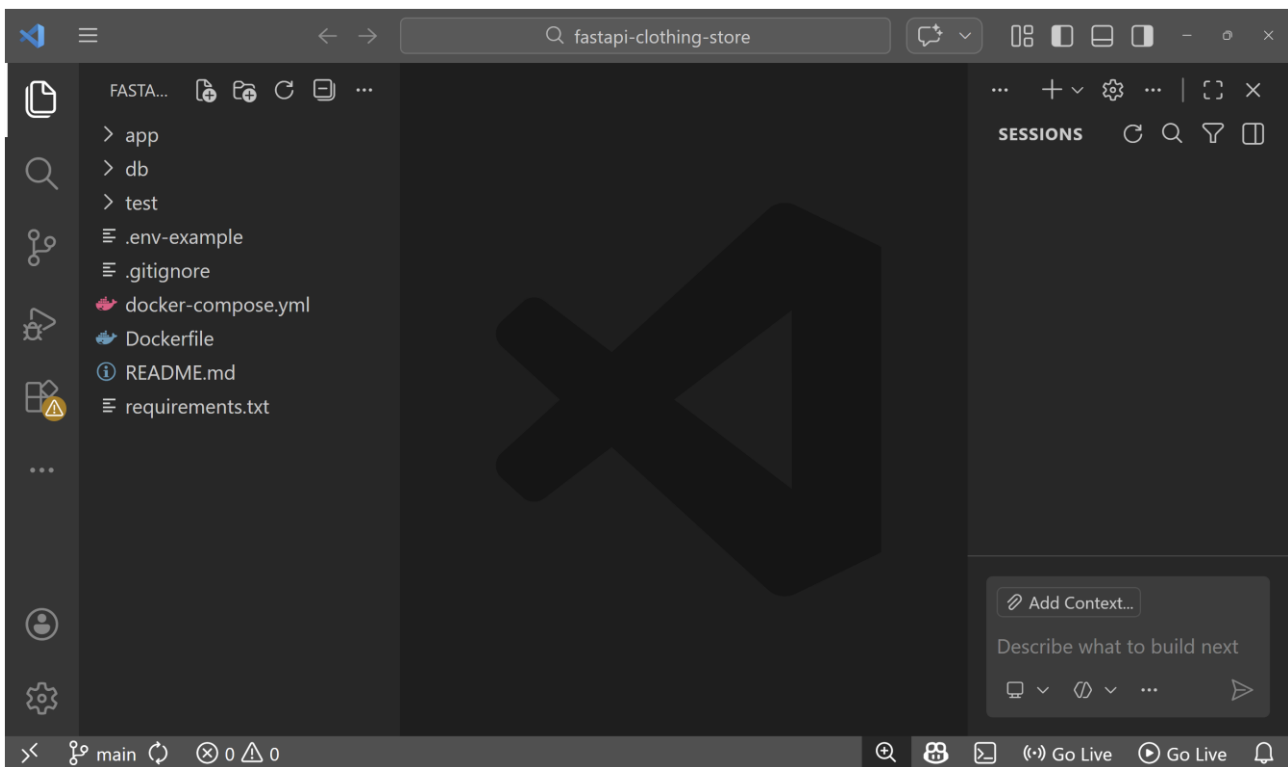
Once the container was running, I opened the FastAPI application in the browser and verified that the root endpoint responded correctly. I also accessed the Swagger UI to confirm that the API endpoints were available and loaded without issues.

To ensure database connectivity from the application, I executed a database check from inside the running FastAPI container. The query returned the correct database name and user, confirming that the application was connected to the Neon PostgreSQL database as expected. This verified that the local FastAPI setup and database connection were working correctly and ready for further development and testing.

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

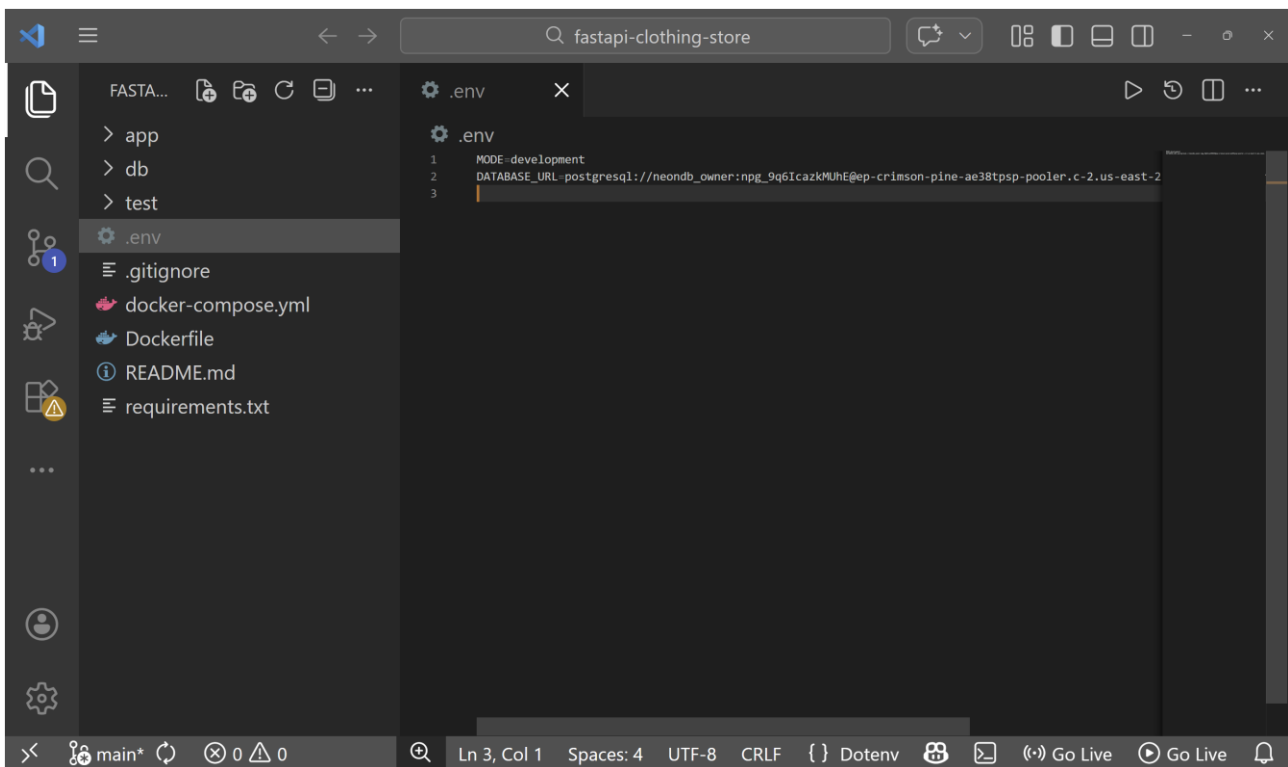
By: Hamed Ahmadinia



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

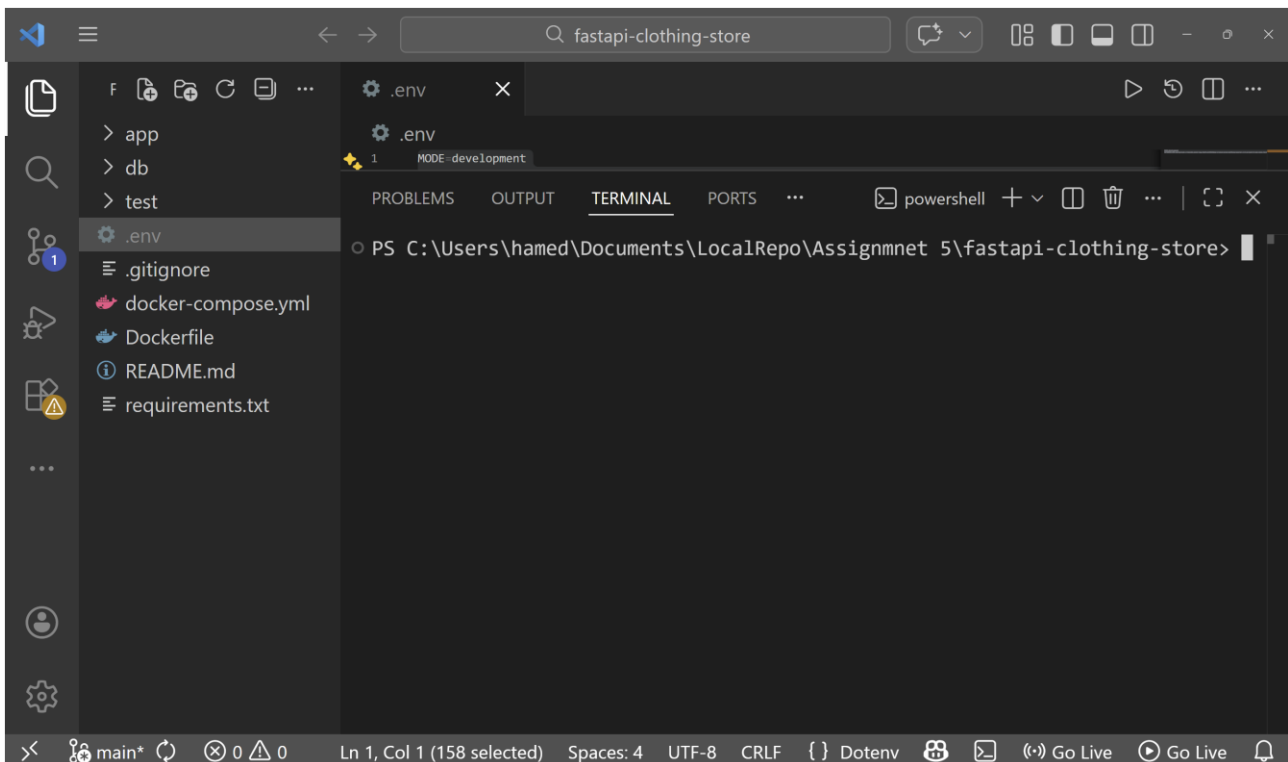
By: Hamed Ahmadinia



The screenshot shows the Visual Studio Code editor interface for a project named 'fastapi-clothing-store'. The left sidebar displays the file explorer with a tree view containing folders 'app', 'db', and 'test', and files '.env', '.gitignore', 'docker-compose.yml', 'Dockerfile', 'README.md', and 'requirements.txt'. The '.env' file is selected and its content is displayed in the main editor area. The content of the '.env' file is as follows:

```
1  MODE=development
2  DATABASE_URL=postgresql://neondb_owner:npg_9q6IcazKMuhE@ep-crimson-pine-ae38tsp-pooler.c-2.us-east-2
3
```

The status bar at the bottom indicates the current file is 'Ln 3, Col 1' with 'Spaces: 4', 'UTF-8' encoding, and 'CRLF' line endings. It also shows 'Dotenv' is configured and 'Go Live' buttons are available.



The screenshot shows the Visual Studio Code editor interface for the 'fastapi-clothing-store' project. The left sidebar is identical to the previous screenshot. The main editor area now shows the 'TERMINAL' tab, which displays the command prompt for a PowerShell session. The command prompt shows the current directory is 'C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothing-store'.

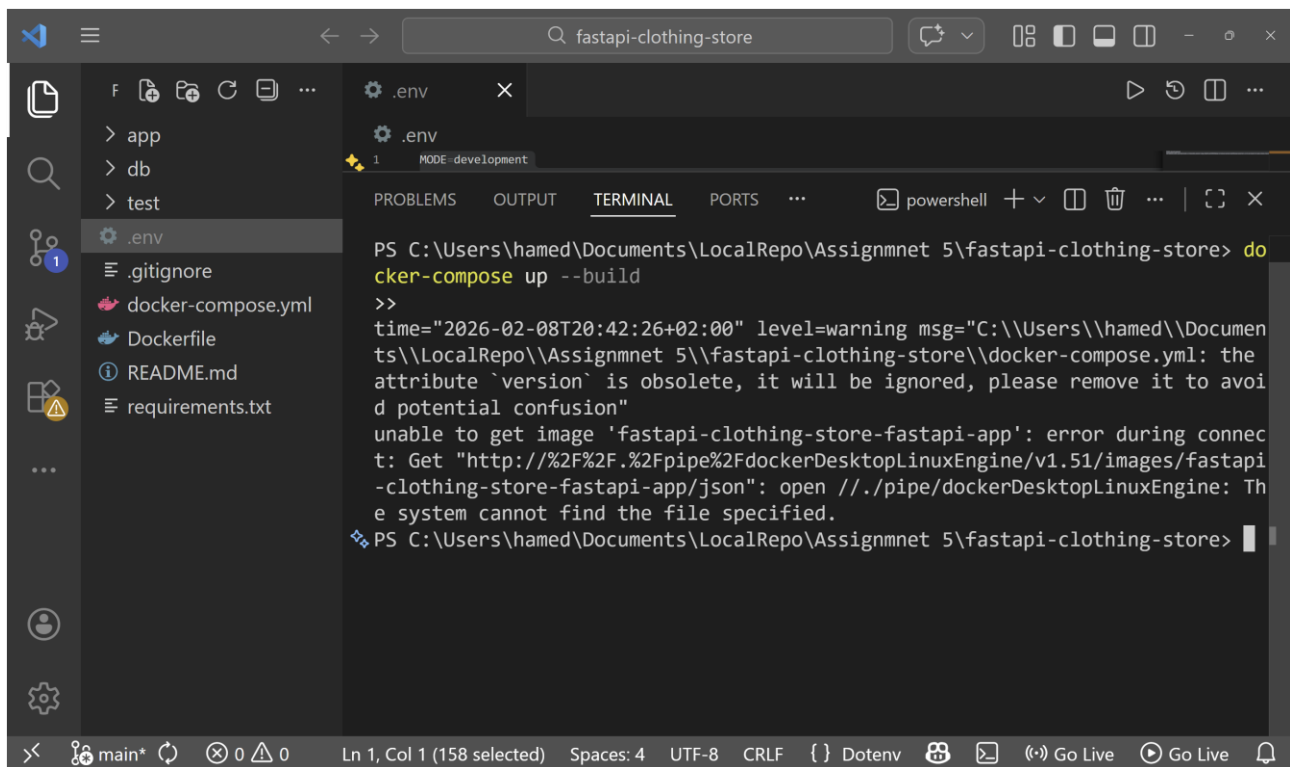
```
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothing-store>
```

The status bar at the bottom indicates the current file is 'Ln 1, Col 1 (158 selected)' with 'Spaces: 4', 'UTF-8' encoding, and 'CRLF' line endings. It also shows 'Dotenv' is configured and 'Go Live' buttons are available.

Assignment 5: Clothing Store API with Analytics

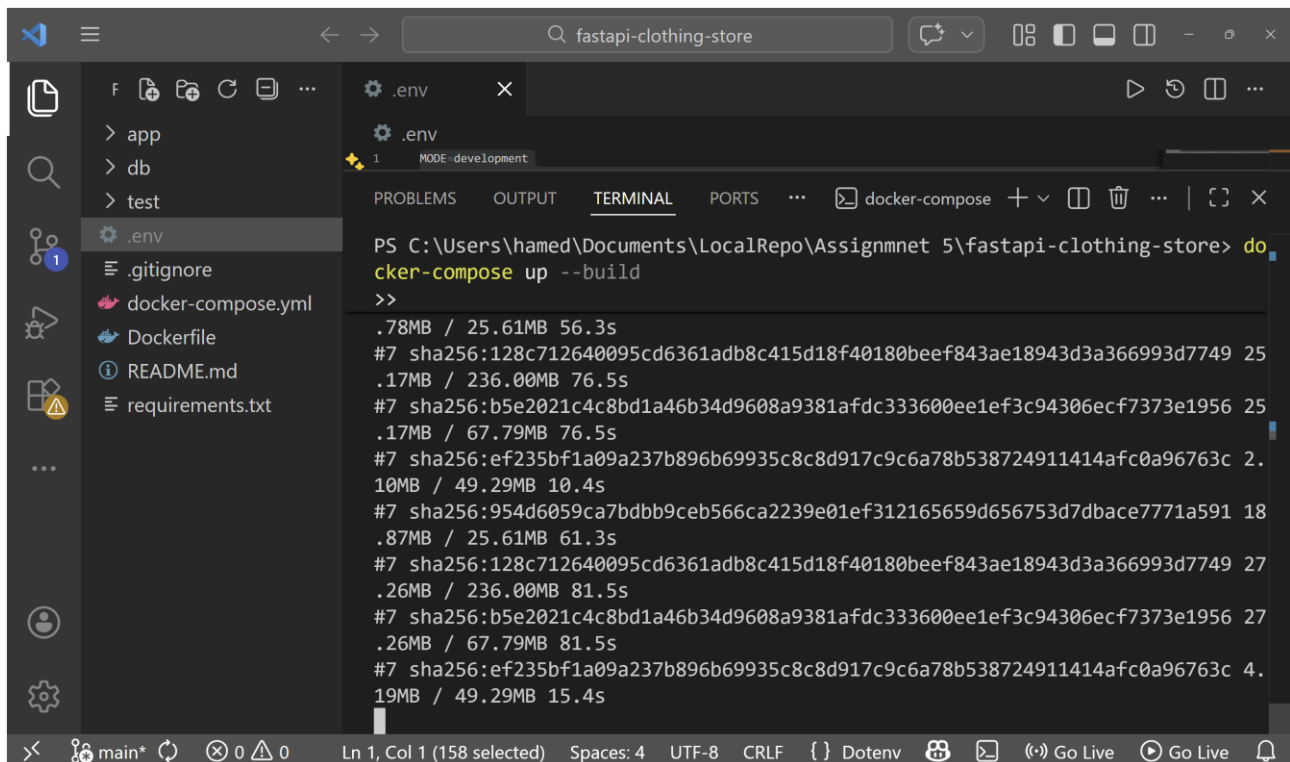
Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the command `docker-compose up --build` and its output. The output indicates a warning about an obsolete 'version' attribute in the `docker-compose.yml` file and a failure to pull the image `fastapi-clothing-store-fastapi-app` due to a connection error from the Docker Desktop Linux Engine.

```
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothing-store> docker-compose up --build
>>
time="2026-02-08T20:42:26+02:00" level=warning msg="C:\\Users\\hamed\\Documents\\LocalRepo\\Assignmnet 5\\fastapi-clothing-store\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
unable to get image 'fastapi-clothing-store-fastapi-app': error during connect: Get "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.51/images/fastapi-clothing-store-fastapi-app/json": open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothing-store>
```



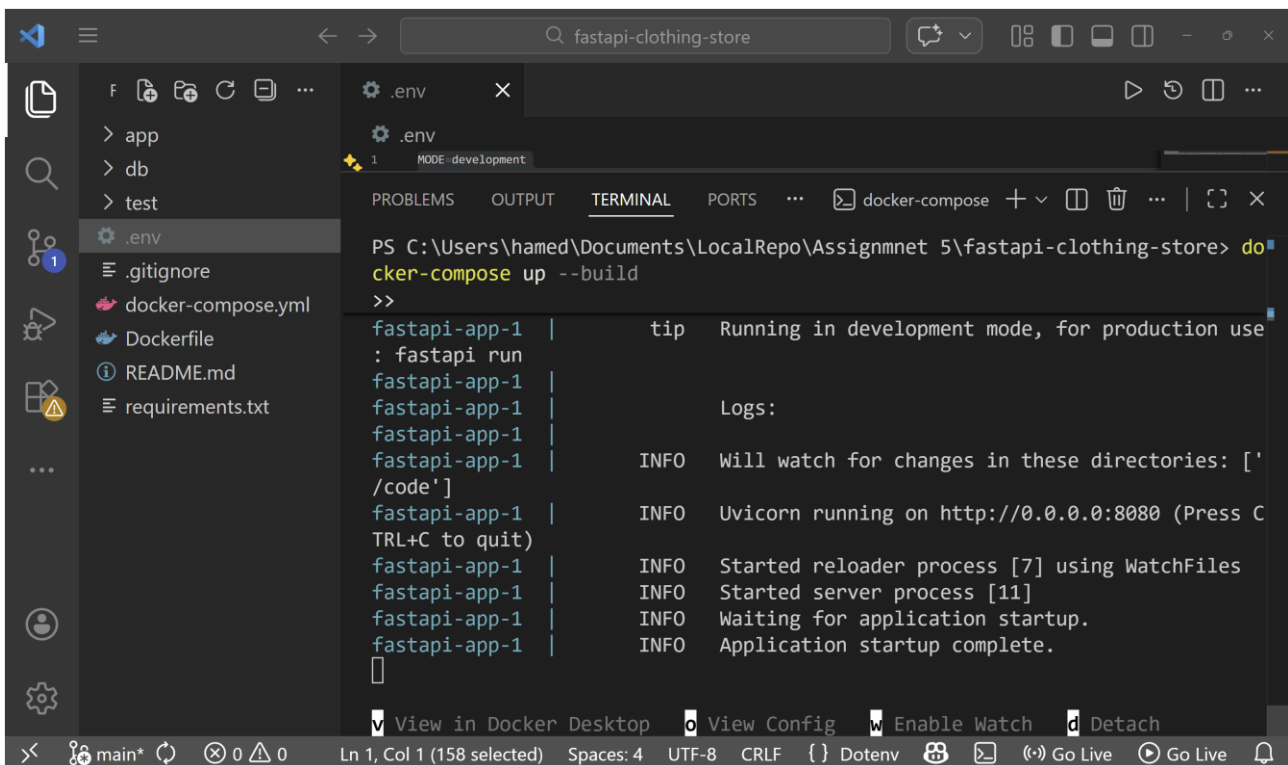
The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the command `docker-compose up --build` and its output. The output shows the successful building and pulling of images for the `fastapi-clothing-store` project, including the `fastapi-clothing-store-fastapi-app` and `fastapi-clothing-store-db` services.

```
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothing-store> docker-compose up --build
>>
.78MB / 25.61MB 56.3s
#7 sha256:128c712640095cd6361adb8c415d18f40180beef843ae18943d3a366993d7749 25
.17MB / 236.00MB 76.5s
#7 sha256:b5e2021c4c8bd1a46b34d9608a9381afdc333600ee1ef3c94306ecf7373e1956 25
.17MB / 67.79MB 76.5s
#7 sha256:ef235bf1a09a237b896b69935c8c8d917c9c6a78b538724911414afc0a96763c 2.
10MB / 49.29MB 10.4s
#7 sha256:954d6059ca7bdbb9ceb566ca2239e01ef312165659d656753d7dbace7771a591 18
.87MB / 25.61MB 61.3s
#7 sha256:128c712640095cd6361adb8c415d18f40180beef843ae18943d3a366993d7749 27
.26MB / 236.00MB 81.5s
#7 sha256:b5e2021c4c8bd1a46b34d9608a9381afdc333600ee1ef3c94306ecf7373e1956 27
.26MB / 67.79MB 81.5s
#7 sha256:ef235bf1a09a237b896b69935c8c8d917c9c6a78b538724911414afc0a96763c 4.
19MB / 49.29MB 15.4s
```

Assignment 5: Clothing Store API with Analytics

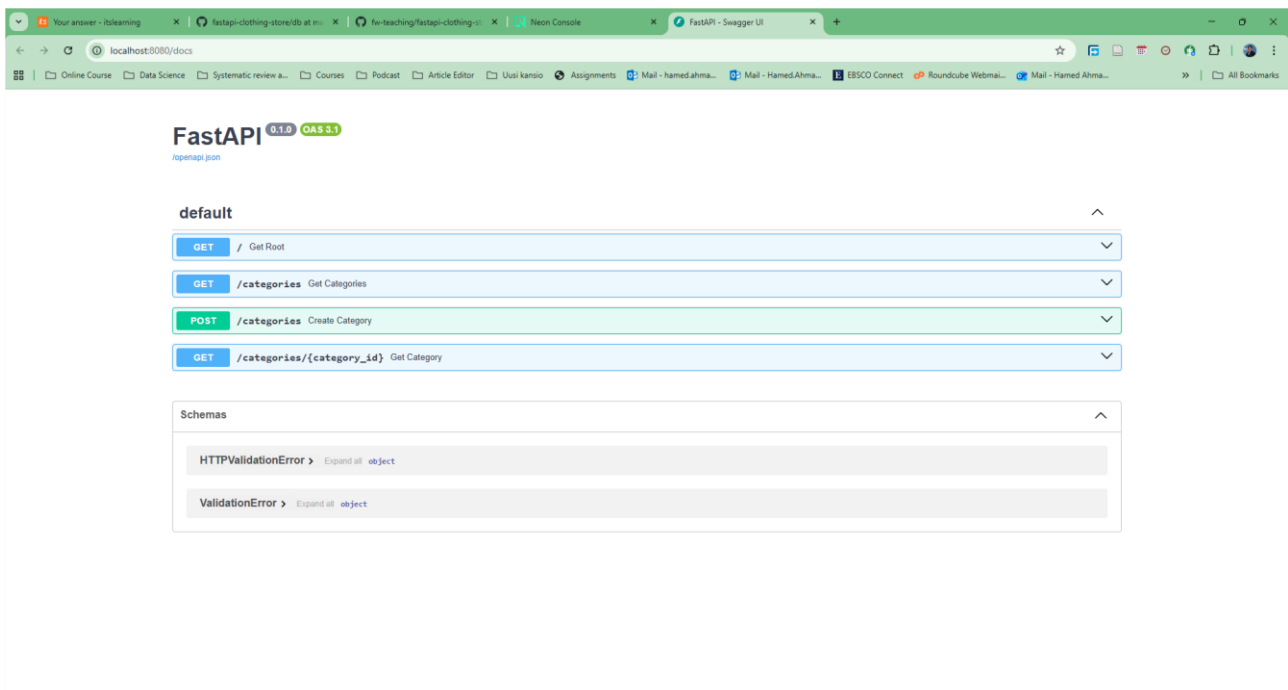
Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the output of the command `docker-compose up --build`. The output shows that the application is running in development mode, and the logs indicate that the server is up and running on `http://0.0.0.0:8080`. The logs also show that the application is watching for changes in the `/code` directory and that the server process has started.

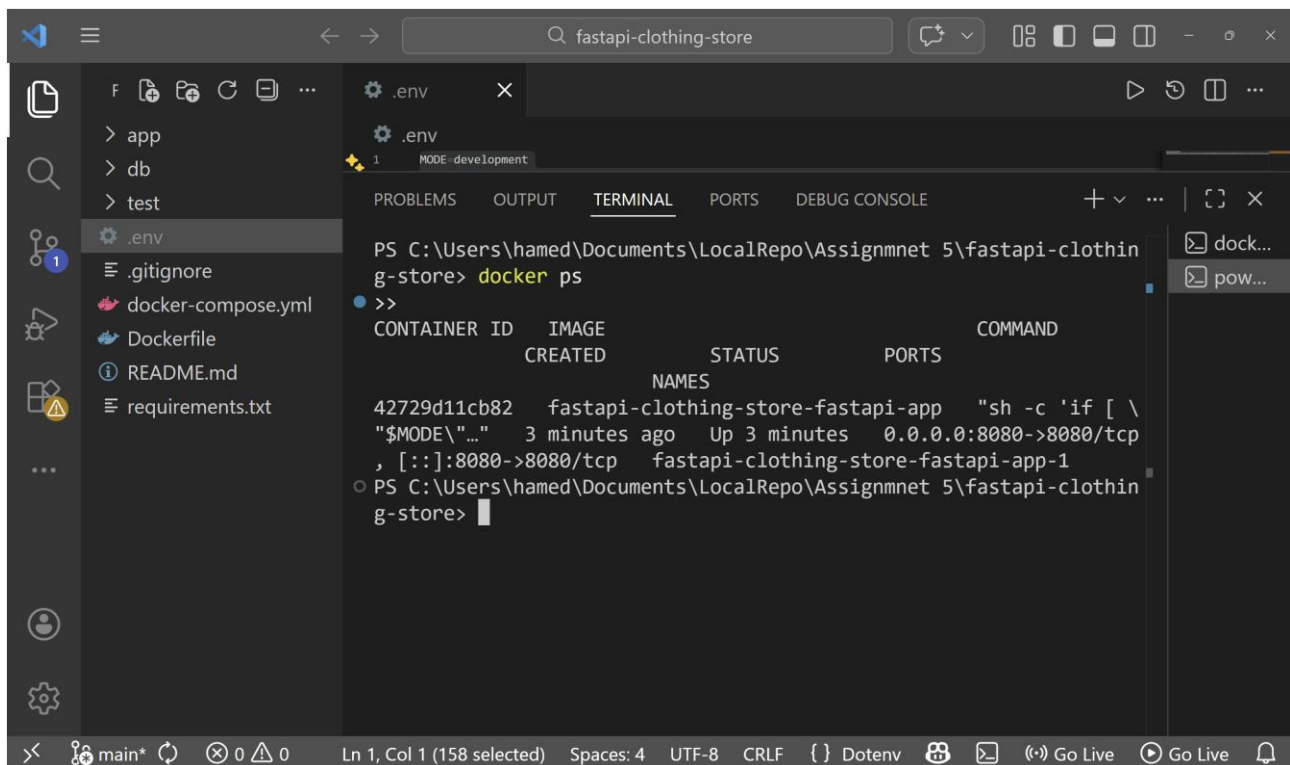
```
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothing-store> docker-compose up --build
>>
fastapi-app-1 | tip Running in development mode, for production use
fastapi-app-1 |
fastapi-app-1 | Logs:
fastapi-app-1 |
fastapi-app-1 | INFO Will watch for changes in these directories: ['/code']
fastapi-app-1 | INFO Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
fastapi-app-1 | INFO Started reloader process [7] using WatchFiles
fastapi-app-1 | INFO Started server process [11]
fastapi-app-1 | INFO Waiting for application startup.
fastapi-app-1 | INFO Application startup complete.
```



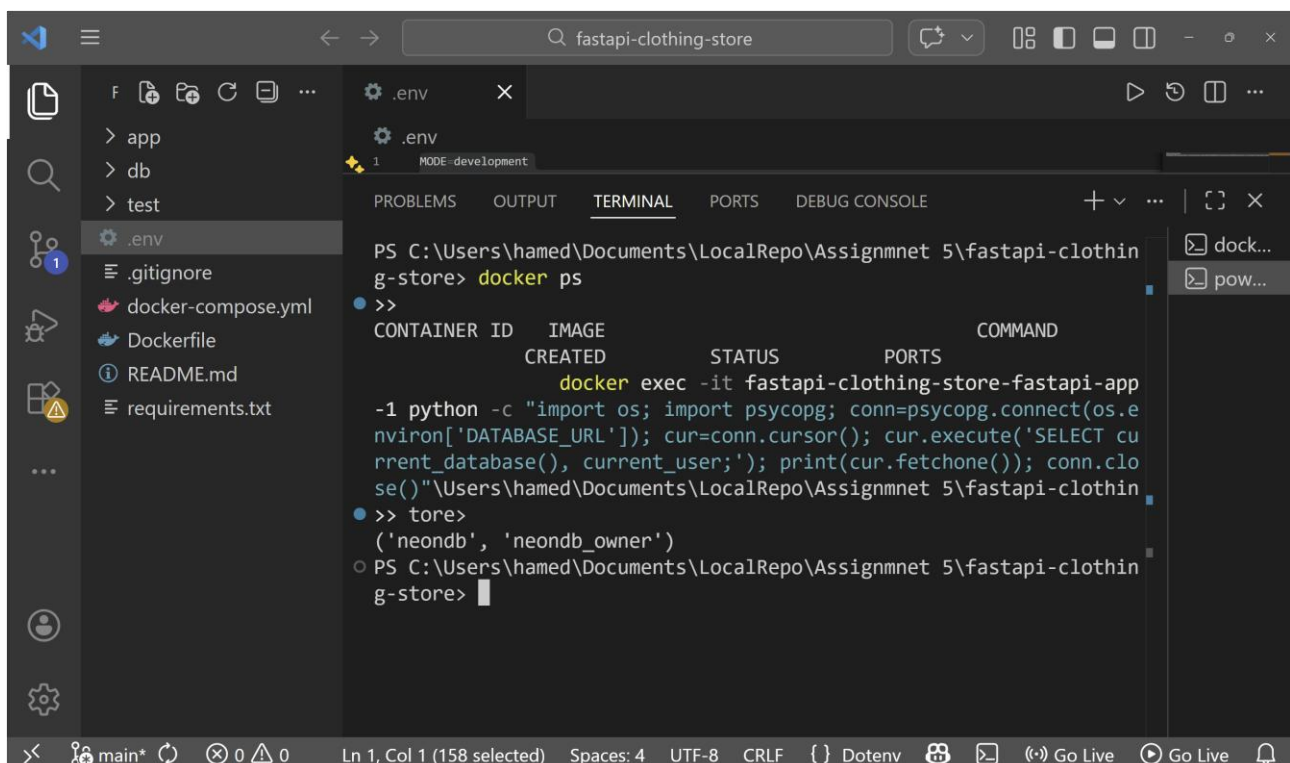
Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



```
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothin
g-store> docker ps
>>
CONTAINER ID   IMAGE                                STATUS      PORTS
42729d11cb82   fastapi-clothing-store-fastapi-app  Up 3 minutes  0.0.0.0:8080->8080/tcp
fastapi-clothing-store-fastapi-app-1
```



```
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothin
g-store> docker ps
>>
CONTAINER ID   IMAGE                                STATUS      PORTS
42729d11cb82   fastapi-clothing-store-fastapi-app  Up 3 minutes  0.0.0.0:8080->8080/tcp
fastapi-clothing-store-fastapi-app-1
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothin
g-store> docker exec -it fastapi-clothing-store-fastapi-app
-1 python -c "import os; import psycopg2; conn=psycopg2.connect(os.e
nviron['DATABASE_URL']); cur=conn.cursor(); cur.execute('SELECT cu
rrent_database(), current_user;'); print(cur.fetchone()); conn.clo
se()"
('neondb', 'neondb_owner')
```

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

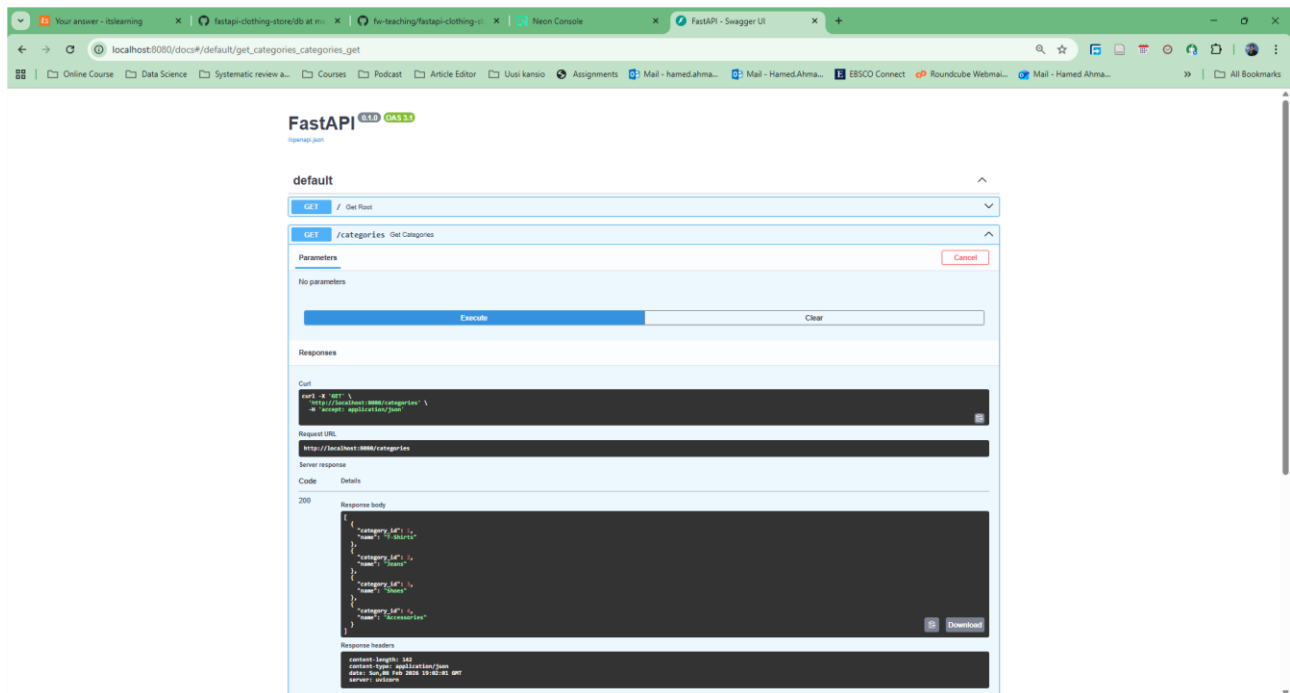
By: Hamed Ahmadinia

Step 4 — Implement GET /products (JOIN)

In this step, I implemented the `GET /products` endpoint to retrieve product data together with its related category information. Since products and categories are stored in separate tables, I used a SQL `JOIN` to combine data from both tables in a single query.

The query joins the `products` table with the `categories` table using the shared `category_id` field. This allows each product to be returned along with its category name instead of only the category ID. The query was executed through the database connection used by the FastAPI application. The response includes product name, category name, price, and stock level.

After implementing the endpoint, I tested it using the Swagger UI. The endpoint returned a list of products with their associated category details, confirming that the join operation worked correctly. This step demonstrated the use of relational database concepts within the API and ensured that product data is returned in a more meaningful and user-friendly format.



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface. On the left, the 'CONNECTIONS' pane lists 'Neon Clothing Store DB' (postgres). Below it, the 'TABLES, VIEWS, FUNCTIONS' pane shows the 'public' schema with five tables: 'categories', 'customers', 'order_items', 'orders', and 'products'. The main pane displays 'Query #4' with the following SQL:

```
SELECT
1  p.product_id,
2  p.name AS product_name,
3  c.name AS category_name
4  FROM products p
5  JOIN categories c
6  ON p.category_id = c.category_id
7  ORDER BY p.product_id;
```

The 'Result 1' pane shows the query results in a table with columns 'product_id', 'product_name', and 'category_name'.

product_id	product_name	category_name
1	Basic White T-Shirt	T-Shirts
2	Black Hoodie	T-Shirts
3	Graphic Tee	T-Shirts
4	Striped Polo Shirt	T-Shirts
5	Blue Denim Jeans	Jeans
6	Black Slim Jeans	Jeans
7	Grey Chinos	Jeans
8	Running Sneakers	Shoes
9	Leather Shoes	Shoes

The status bar at the bottom indicates the connection is successful and the query finished.

The screenshot shows a code editor with the file 'main.py' open. The code defines a FastAPI application for a clothing store. It includes a database connection, a function to create categories, and a function to get products with a JOIN query.

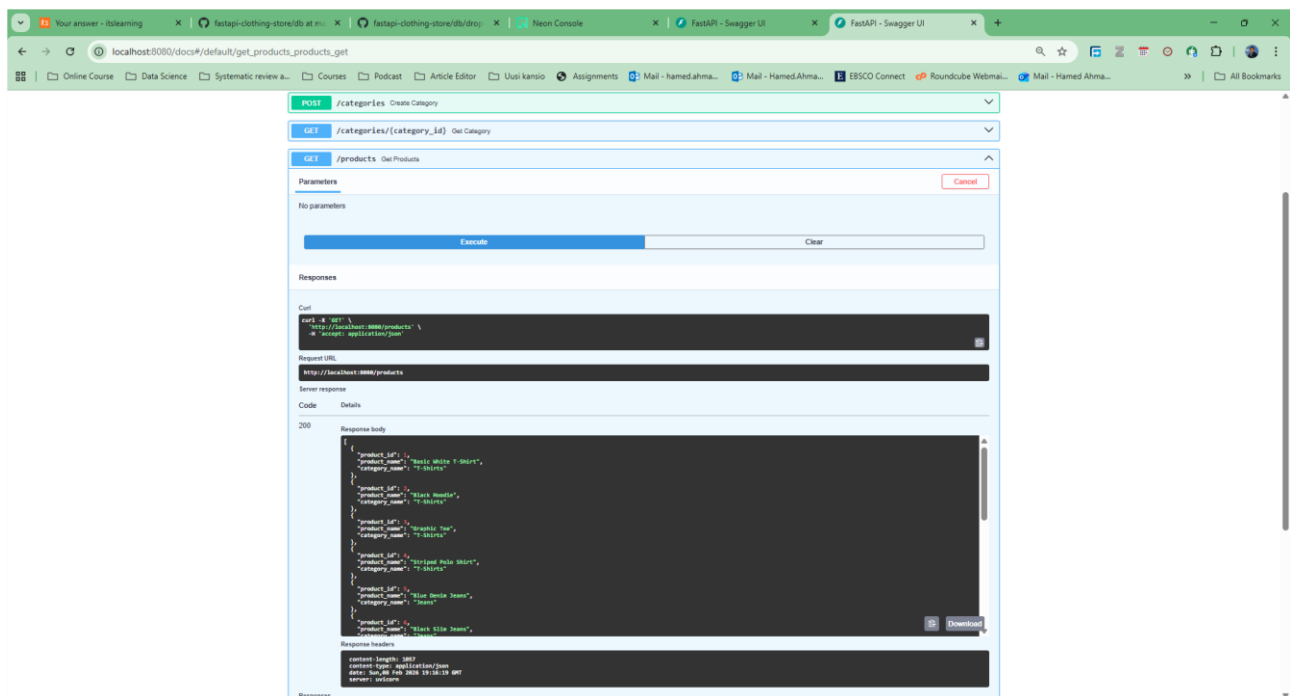
```
35 def create_category(data: dict):
36     with get_conn() as conn, conn.cursor() as cur:
37         cur.execute("INSERT INTO categories (name) VALUES (%s) RETURNING category_id, name;", (name,))
38         return cur.fetchone()
39
40 # GET /products (JOIN products + categories)
41 @app.get("/products")
42 def get_products():
43     query = """
44     SELECT
45         p.product_id,
46         p.name AS product_name,
47         c.name AS category_name
48     FROM products p
49     JOIN categories c
50     ON p.category_id = c.category_id
51     ORDER BY p.product_id;
52     """
53     with get_conn() as conn, conn.cursor() as cur:
54         cur.execute(query)
55         return cur.fetchall()
```

The terminal at the bottom shows the command prompt for the 'fastapi-clothing-store' project.

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



Step 5 — Implement POST /orders with transaction + stock decrement

In this step, I created the POST /orders endpoint to place an order and update stock in the database. The goal was to make sure an order is only saved if every item in the request is valid and there is enough stock for each product.

First, the endpoint reads customer_id and the items list from the request body. I added input checks so the request is rejected if the customer ID is missing or the items list is empty. Then, the code opens a database connection and starts a manual transaction by turning off autocommit.

Inside the transaction, the API inserts a new row into the orders table and stores the new order_id. After that, it loops through each item in the order. For every product, it checks that product_id and quantity are valid, then it locks the product row using SELECT ... FOR UPDATE to prevent two orders from reducing the same stock at the same time. If the product does not exist or if there is not enough stock, the API returns an error and the transaction is rolled back.

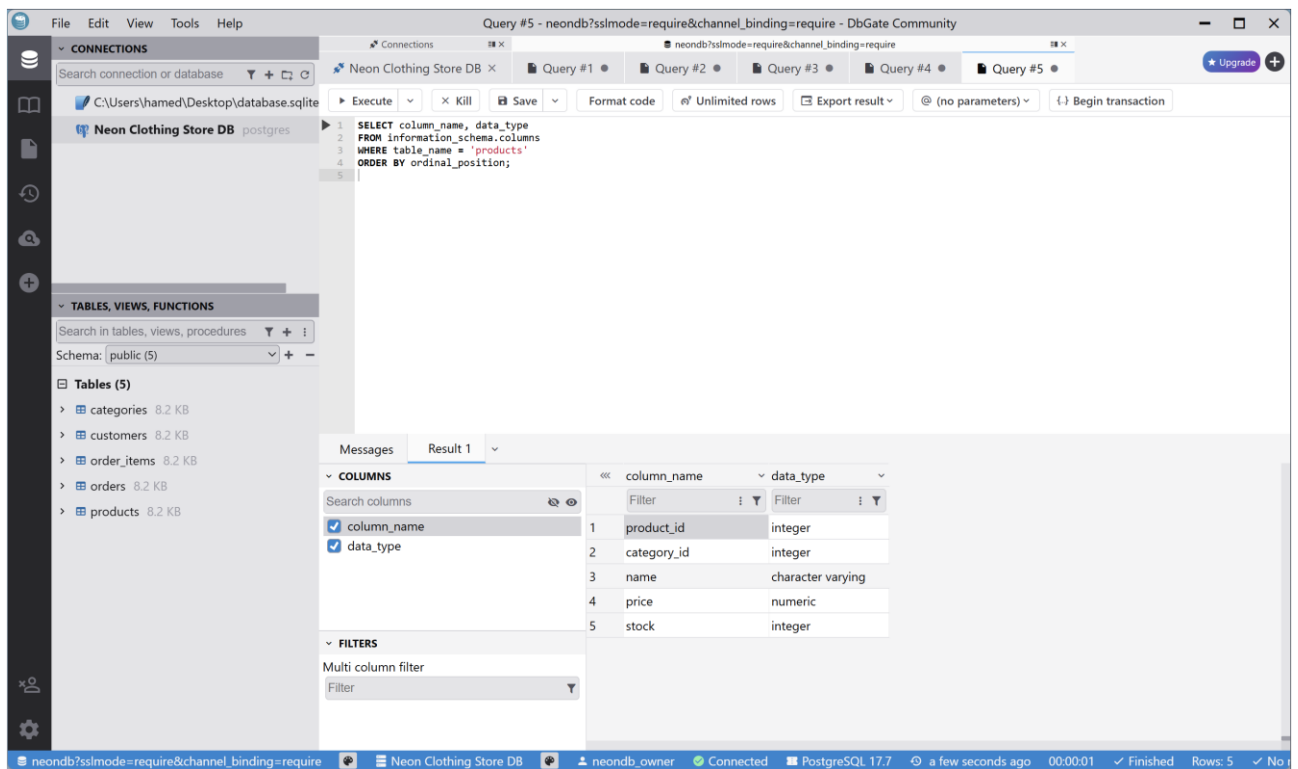
When stock is available, the endpoint updates the products table by subtracting the ordered quantity, and then inserts the item into the order_items table. After all items are processed successfully, the transaction is committed. If any error happens during the process, the transaction is rolled back to avoid partial orders or incorrect stock updates. Although the assignment example shows a single product per request, I implemented support for multiple order items using a list, while preserving the same validation logic.

Finally, I tested the endpoint using Swagger UI with sample orders. Successful requests created an order and reduced product stock correctly, and invalid requests (wrong product ID or insufficient stock) failed without changing the database.

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

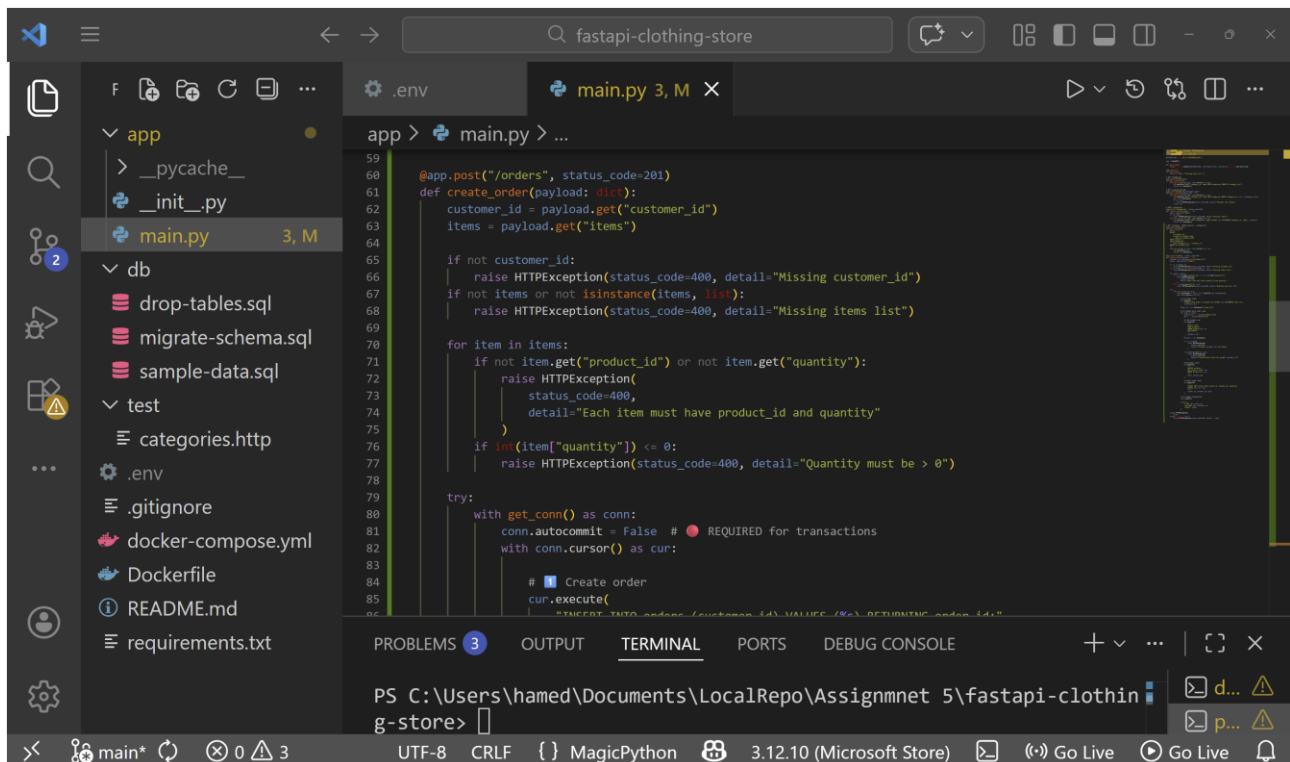


The screenshot shows the DbGate Community interface connected to a PostgreSQL database named 'Neon Clothing Store DB'. A query is executed, showing the columns of the 'products' table. The query is:

```
1 SELECT column_name, data_type
2 FROM information_schema.columns
3 WHERE table_name = 'products'
4 ORDER BY ordinal_position;
5
```

The result shows the following columns:

column_name	data_type
product_id	integer
category_id	integer
name	character varying
price	numeric
stock	integer



The screenshot shows a code editor with the following files in the project:

- app
 - __pycache__
 - __init__.py
 - main.py (3, M)
- db
 - drop-tables.sql
 - migrate-schema.sql
 - sample-data.sql
- test
 - categories.http
- Other files: .env, .gitignore, docker-compose.yml, Dockerfile, README.md, requirements.txt

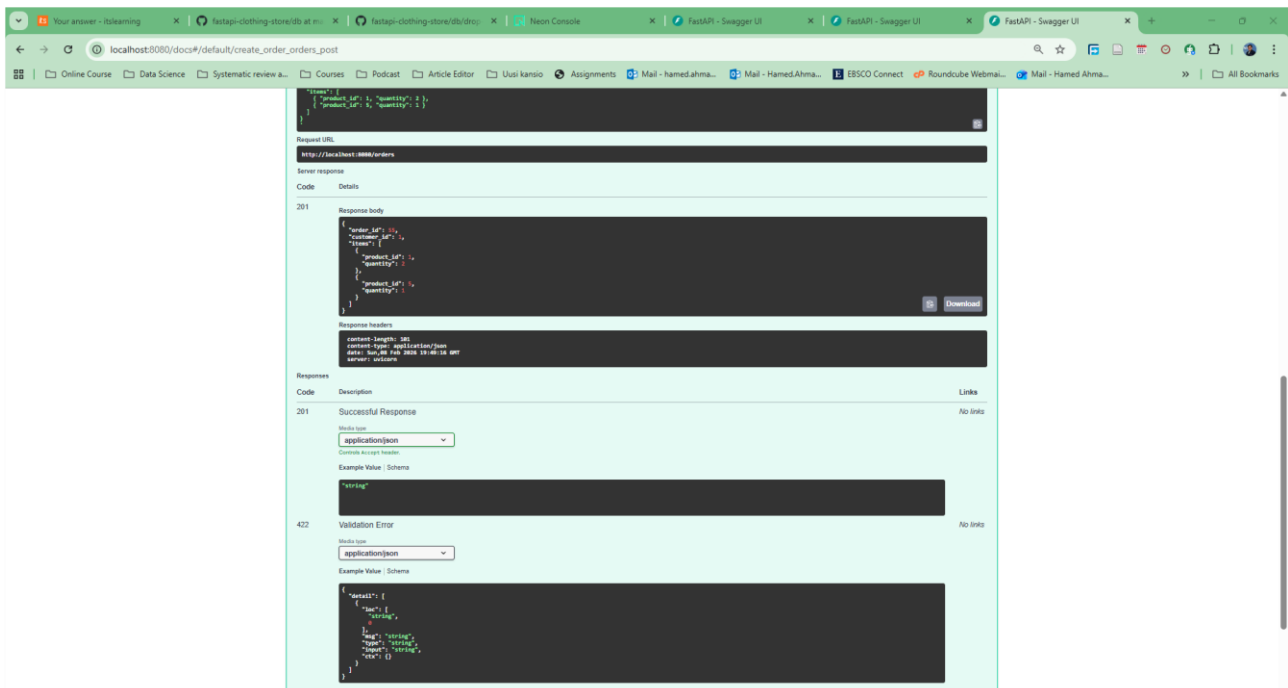
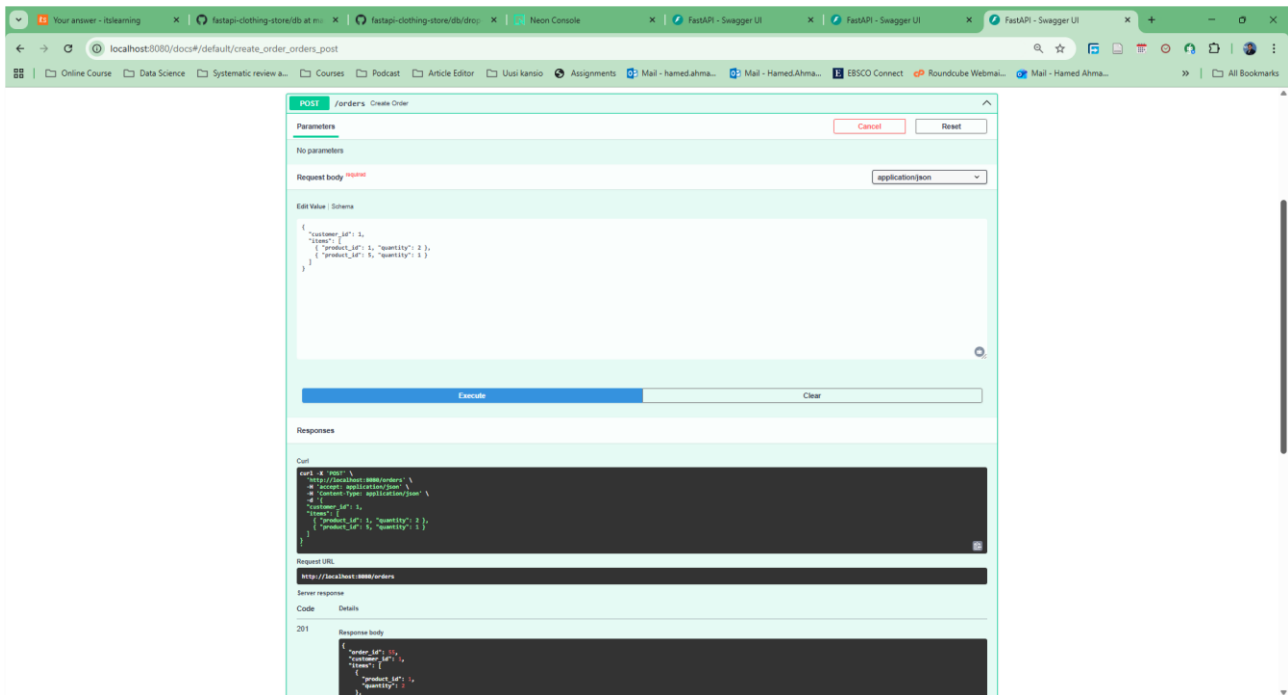
The main.py file contains the following code:

```
59 @app.post("/orders", status_code=201)
60 def create_order(payload: dict):
61     customer_id = payload.get("customer_id")
62     items = payload.get("items")
63
64     if not customer_id:
65         raise HTTPException(status_code=400, detail="Missing customer_id")
66     if not items or not isinstance(items, list):
67         raise HTTPException(status_code=400, detail="Missing items list")
68
69     for item in items:
70         if not item.get("product_id") or not item.get("quantity"):
71             raise HTTPException(
72                 status_code=400,
73                 detail="Each item must have product_id and quantity"
74             )
75         if len(item["quantity"]) <= 0:
76             raise HTTPException(status_code=400, detail="Quantity must be > 0")
77
78     try:
79         with get_conn() as conn:
80             conn.autocommit = False # REQUIRED for transactions
81             with conn.cursor() as cur:
82                 # Create order
83                 cur.execute(
84                     "INSERT INTO orders (customer_id, product_id, quantity) VALUES (%s, %s, %s)"
85                 )
```

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface. The left sidebar displays the 'Neon Clothing Store DB' connection and a list of tables: categories, customers, order_items, orders, and products. The main window shows Query #6 with the following SQL:

```
1 SELECT product_id, stock
2 FROM products
3 WHERE product_id = 1;
```

The 'Result 1' tab is active, showing a table with two columns: product_id and stock. The data is as follows:

product_id	stock
1	98

The status bar at the bottom indicates the query is finished, with 1 row returned.

The screenshot shows the DbGate Community interface. The left sidebar displays the 'Neon Clothing Store DB' connection and a list of tables: categories, customers, order_items, orders, and products. The main window shows Query #7 with the following SQL:

```
1 SELECT product_id, name, stock
2 FROM products
3 WHERE product_id IN (1, 5)
4 ORDER BY product_id;
```

The 'Result 1' tab is active, showing a table with three columns: product_id, name, and stock. The data is as follows:

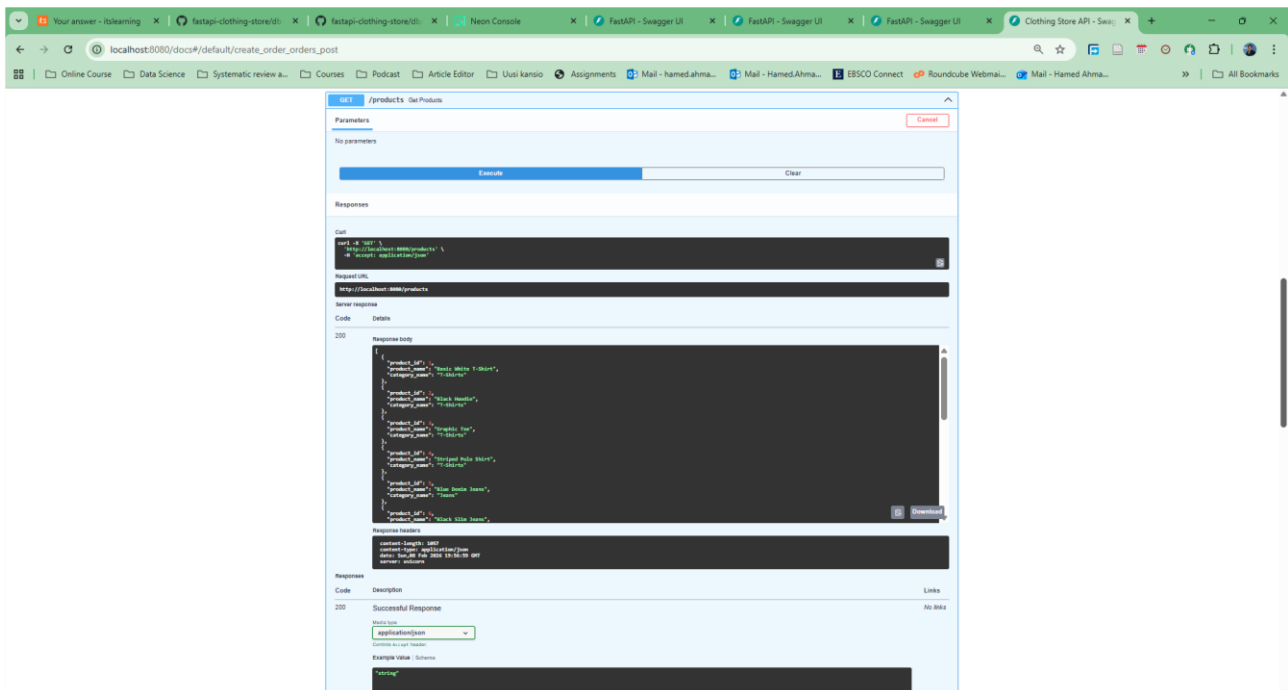
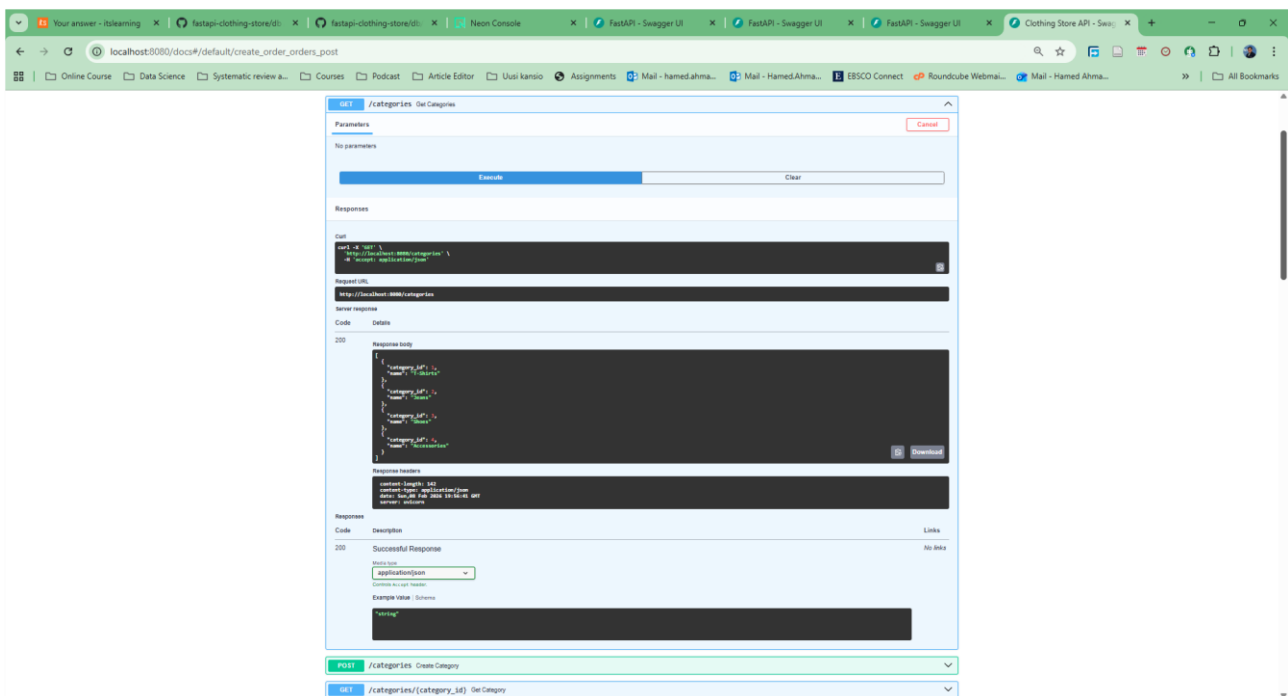
product_id	name	stock
1	Basic White T-Shirt	98
5	Blue Denim Jeans	49

The status bar at the bottom indicates the query is finished, with 2 rows returned.

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

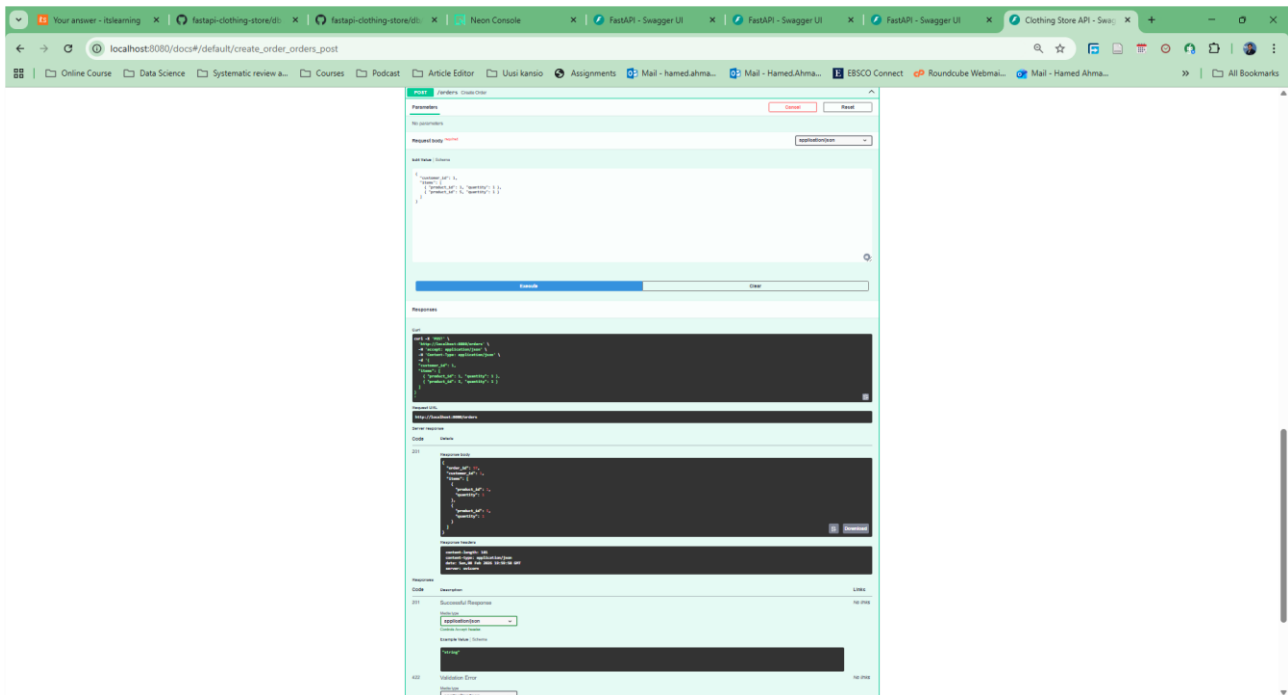
By: Hamed Ahmadiania



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania



Step 6 — Analytics endpoints (SQL aggregates + joins)

In this step, I added analytics endpoints that return summary statistics from the database using SQL aggregates and joins. The purpose was to practice JOIN, GROUP BY, and aggregate functions like SUM() and COUNT() to produce useful reports from the order data.

First, I implemented GET /analytics/top-products to show the best-selling products. This endpoint joins order_items with products, groups the results by product, and calculates total_units_sold using SUM(oi.quantity). The results are sorted from highest to lowest and limited to the top five products.

Next, I implemented GET /analytics/category-sales to summarize sales by category. This query joins order_items → products → categories, groups by category, and sums total units sold per category. The output shows which categories have the most sales overall.

Finally, I implemented GET /analytics/top-customers to find the most active customers. This endpoint joins orders with customers, groups by customer, and counts how many orders each customer has made using COUNT(o.order_id). The results are sorted in descending order and limited to the top ten customers.

All three endpoints reuse the same database connection helper and return the query results as JSON in Swagger UI, which confirmed that the aggregate queries and joins were working correctly. These endpoints fulfill the required statistics functionality, although the route names differ slightly and include additional analytical breakdowns.

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot shows the DbGate Community interface for Query #8. The query is a SQL statement that joins the 'products' and 'order_items' tables to calculate the total units sold for each product, ordered by total units sold in descending order and limited to 5 results.

```
1 SELECT p.product_id, p.name AS product_name, SUM(oi.quantity) AS total_units_sold
2 FROM order_items oi
3 JOIN products p ON p.product_id = oi.product_id
4 GROUP BY p.product_id, p.name
5 ORDER BY total_units_sold DESC
6 LIMIT 5;
```

The result set shows the following data:

product_id	product_name	total_units_sold
5	Blue Denim Jeans	22
1	Basic White T-Shirt	21
8	Running Sneakers	20
7	Grey Chinos	20
9	Leather Shoes	19

The screenshot shows the DbGate Community interface for Query #10. The query is a SQL statement that joins the 'customers' and 'orders' tables to calculate the total number of orders for each customer, ordered by total orders in descending order and limited to 6 results.

```
1 SELECT
2   cu.customer_id,
3   cu.first_name || ' ' || cu.last_name AS customer_name,
4   COUNT(o.order_id) AS total_orders
5 FROM orders o
6 JOIN customers cu ON cu.customer_id = o.customer_id
7 GROUP BY cu.customer_id, cu.first_name, cu.last_name
8 ORDER BY total_orders DESC
9 LIMIT 6;
```

The result set shows the following data:

customer_id	customer_name	total_orders
1	Alice Johnson	12
5	Eva Wilson	10
4	David Brown	9
3	Carol Davis	9
2	Bob Smith	9
6	Frank Miller	8

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

Query #9 - neondb?sslmode=require&channel_binding=require - DbGate Community

```
1 SELECT c.category_id, c.name AS category_name, SUM(oi.quantity) AS total_units_sold
2 FROM order_items oi
3 JOIN products p ON p.product_id = oi.product_id
4 JOIN categories c ON c.category_id = p.category_id
5 GROUP BY c.category_id, c.name
6 ORDER BY total_units_sold DESC;
```

Result 1

category_id	category_name	total_units_sold
1	T-Shirts	69
2	Jeans	60
3	Shoes	57
4	Accessories	52

Query #11 - neondb?sslmode=require&channel_binding=require - DbGate Community

```
1 SELECT
2 cu.customer_id,
3 cu.email AS customer_name,
4 COUNT(o.order_id) AS total_orders
5 FROM orders o
6 JOIN customers cu ON cu.customer_id = o.customer_id
7 GROUP BY cu.customer_id, cu.email
8 ORDER BY total_orders DESC
9 LIMIT 10;
```

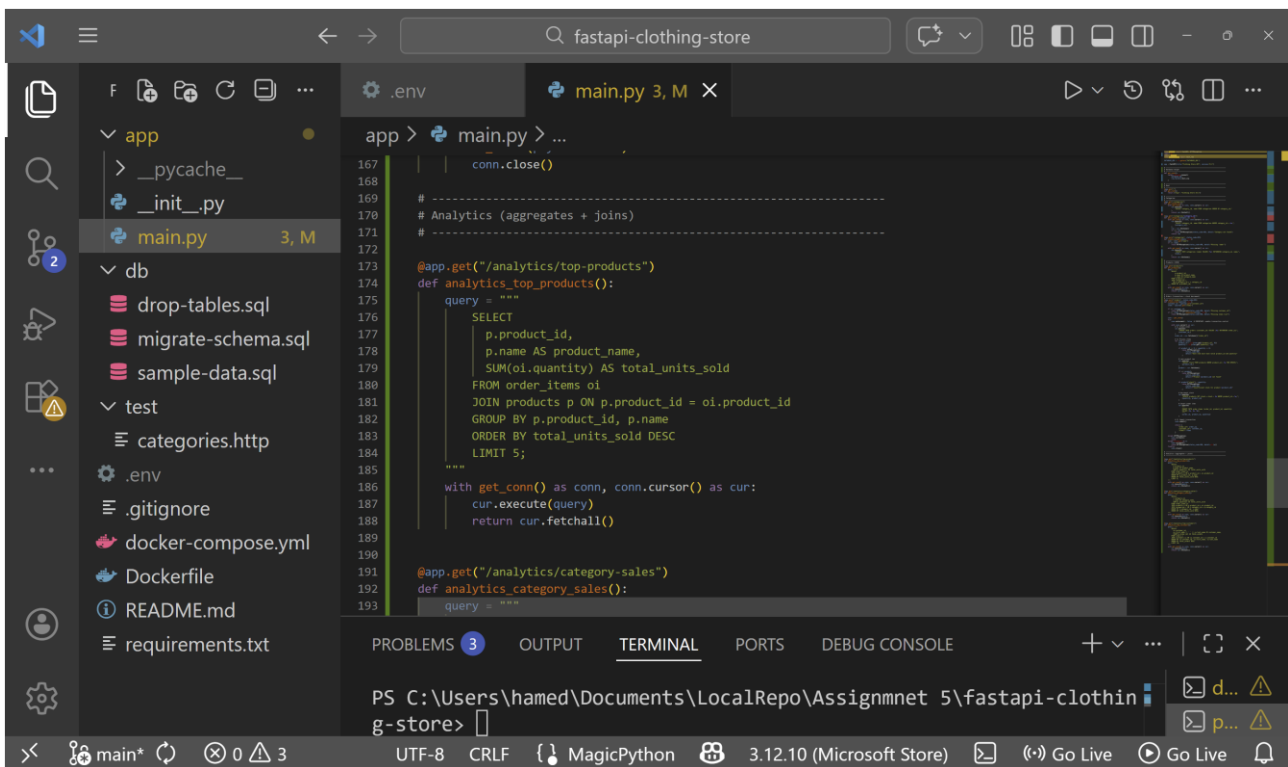
Result 1

customer_id	customer_name	total_orders
1	alice@example.com	12
2	eva@example.com	10
3	david@example.com	9
4	carol@example.com	9
5	bob@example.com	9
6	frank@example.com	8

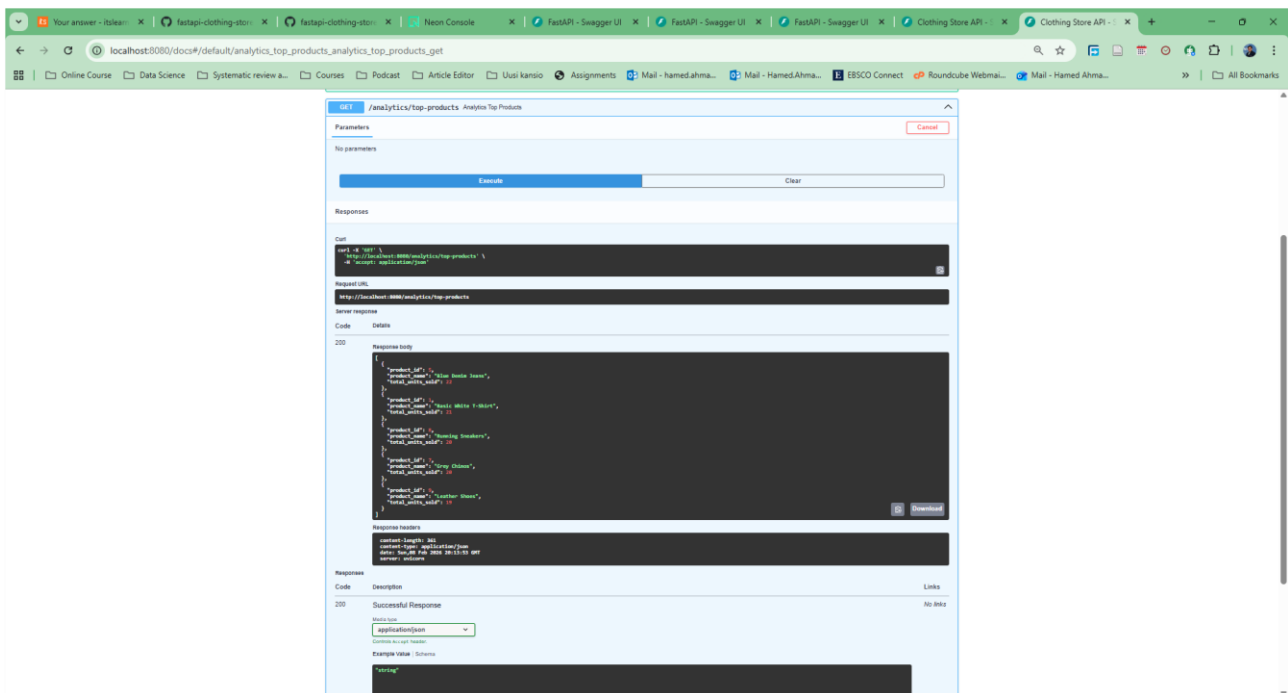
Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadiania



```
167 | conn.close()
168 |
169 | # -----
170 | # Analytics (aggregates + joins)
171 | # -----
172 |
173 | @app.get("/analytics/top-products")
174 | def analytics_top_products():
175 |     query = """
176 |     SELECT
177 |         p.product_id,
178 |         p.name AS product_name,
179 |         SUM(oi.quantity) AS total_units_sold
180 |     FROM order_items oi
181 |     JOIN products p ON p.product_id = oi.product_id
182 |     GROUP BY p.product_id, p.name
183 |     ORDER BY total_units_sold DESC
184 |     LIMIT 5;
185 |     """
186 |     with get_conn() as conn, conn.cursor() as cur:
187 |         cur.execute(query)
188 |         return cur.fetchall()
189 |
190 |
191 | @app.get("/analytics/category-sales")
192 | def analytics_category_sales():
193 |     query = """
```



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

The screenshot displays the Swagger UI for the Clothing Store API. The selected endpoint is `GET /analytics/category-sales` with the description "Analytics Category Sales". There are no parameters for this endpoint. The response is a 200 status code with a JSON body. The response body is a list of objects, each representing a category sale. The response headers include `Content-Type: application/json` and `Content-Length: 200`. The response is a successful response.

```
curl -X GET http://localhost:8080/api/analytics/category-sales -H 'accept: application/json'
```

```
{
  "category_id": 1,
  "category_name": "T-Shirts",
  "total_sales": 100,
  "total_price": 1000,
  "category_id": 2,
  "category_name": "Hoodies",
  "total_sales": 50,
  "total_price": 500,
  "category_id": 3,
  "category_name": "Jeans",
  "total_sales": 75,
  "total_price": 750,
  "category_id": 4,
  "category_name": "Accessories",
  "total_sales": 25,
  "total_price": 250
}
```

Response Headers:

```
Content-Length: 200
Content-Type: application/json
Date: Wed, 01 Nov 2023 08:12:00 GMT
Server: Express
```

Response: 200 Successful Response

Media type: application/json

Example Value: Schema

The screenshot displays the Swagger UI for the Clothing Store API. The selected endpoint is `GET /analytics/top-customers` with the description "Analytics Top Customers". There are no parameters for this endpoint. The response is a 200 status code with a JSON body. The response body is a list of objects, each representing a top customer. The response headers include `Content-Type: application/json` and `Content-Length: 200`. The response is a successful response.

```
curl -X GET http://localhost:8080/api/analytics/top-customers -H 'accept: application/json'
```

```
{
  "customer_id": 1,
  "customer_name": "Alice Johnson",
  "total_sales": 100,
  "total_price": 1000,
  "customer_id": 2,
  "customer_name": "Bob Wilson",
  "total_sales": 50,
  "total_price": 500,
  "customer_id": 3,
  "customer_name": "David Brown",
  "total_sales": 75,
  "total_price": 750,
  "customer_id": 4,
  "customer_name": "Carol Davis",
  "total_sales": 25,
  "total_price": 250,
  "customer_id": 5,
  "customer_name": "Eve Smith",
  "total_sales": 10,
  "total_price": 100
}
```

Response Headers:

```
Content-Length: 200
Content-Type: application/json
Date: Wed, 01 Nov 2023 08:12:00 GMT
Server: Express
```

Response: 200 Successful Response

Media type: application/json

Example Value: Schema

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia

Step 7 (Optional) — Auth (bcrypt + JWT) + RBAC

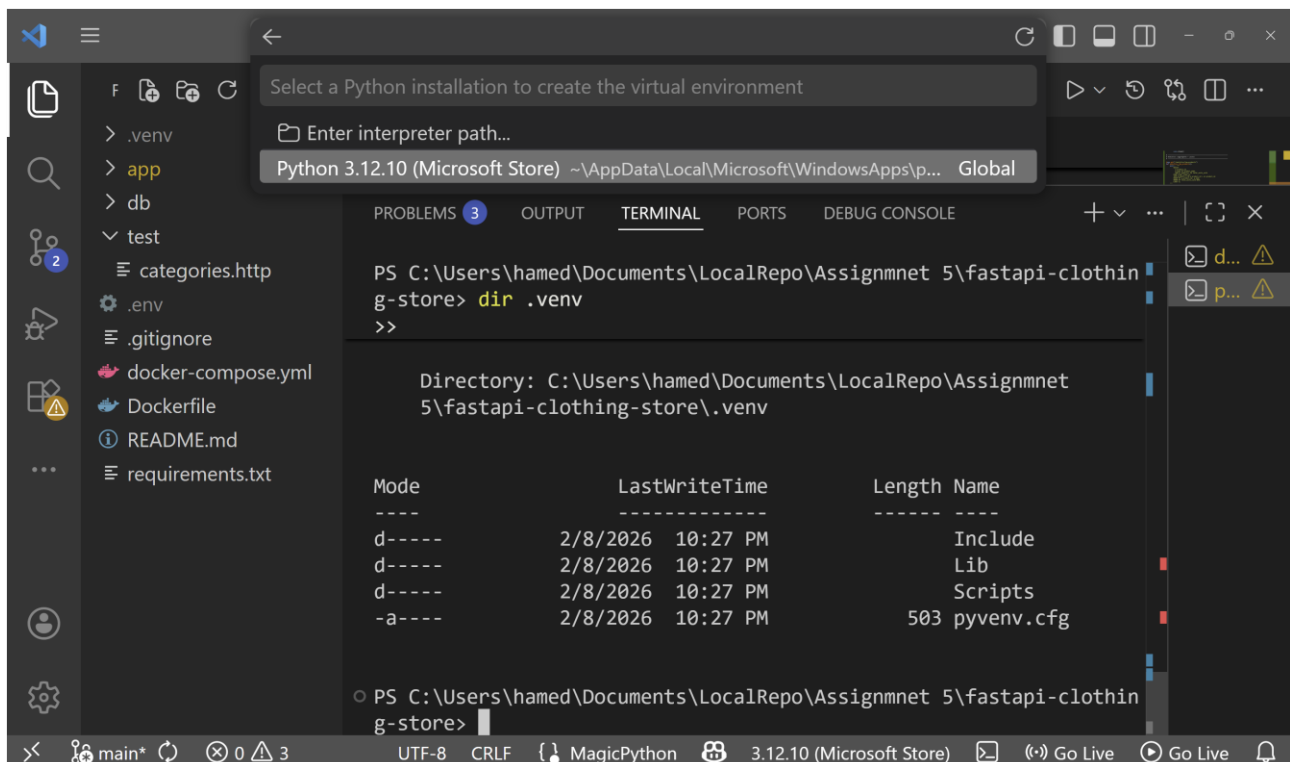
In this step, I added authentication and access control to the Clothing Store API. The goal was to securely register users, allow them to log in, and restrict certain endpoints based on user roles.

I first implemented user registration using the `/auth/register` endpoint. When a user registers, the password is not stored directly in the database. Instead, it is hashed using bcrypt before being saved. I verified this by checking the customers table in DbGate and confirming that the password was stored as a hash.

Next, I implemented user login with the `/auth/login` endpoint. Users log in using their email and password. If the credentials are correct, the API returns a JSON Web Token (JWT). This token is later used to access protected endpoints. I tested this in Swagger and confirmed that a token was returned after a successful login.

Finally, I added role-based access control. Each user has a role stored in the database (user or admin). Admin-only endpoints, such as statistics and administrative CRUD operations, were protected so that only users with the admin role could access them. I confirmed this by testing the endpoints with and without admin privileges and by checking the user role in DbGate.

This completed the authentication and authorization setup for the API.



The screenshot shows a VS Code interface with a terminal window open. The terminal is running a PowerShell command to create a virtual environment. A dropdown menu is visible, showing the selection of Python 3.12.10 (Microsoft Store) as the interpreter. The terminal output shows the directory structure of the virtual environment.

```
PS C:\Users\hamed\Documents\LocalRepo\Assignmnet 5\fastapi-clothin
g-store> dir .venv
>>

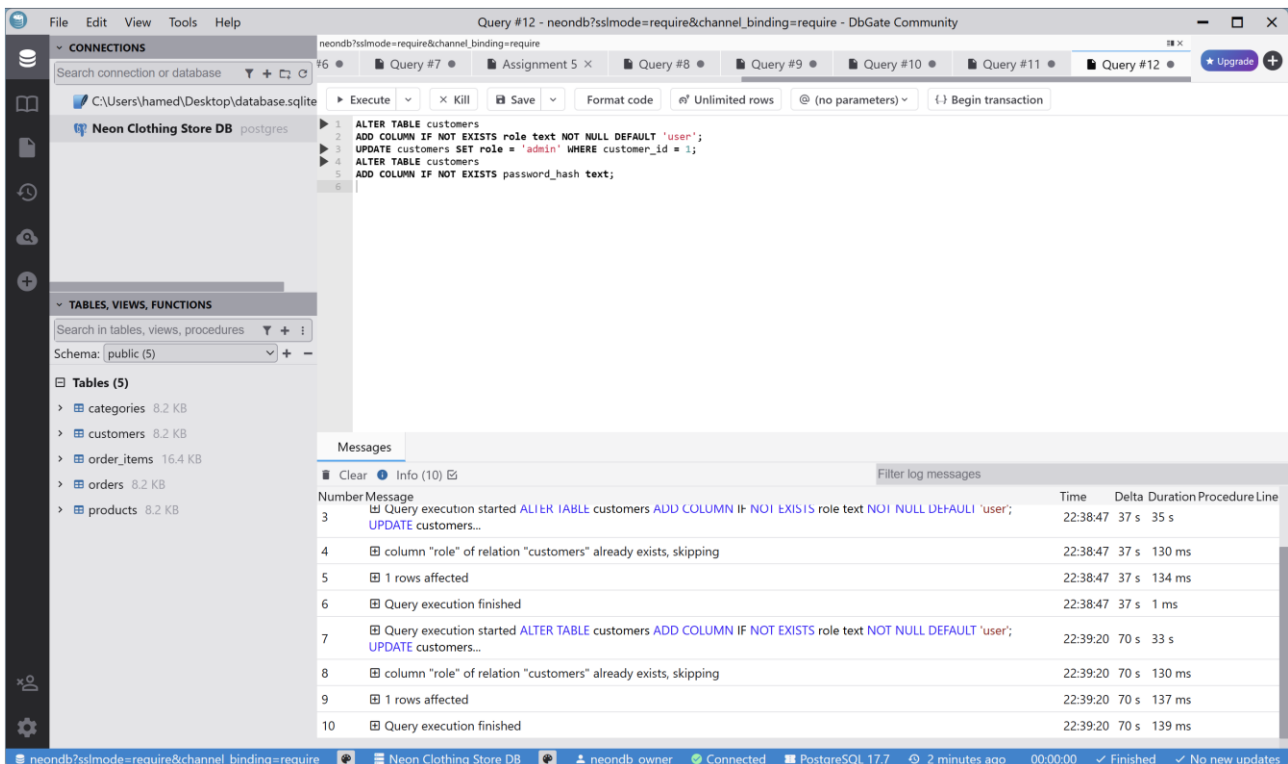
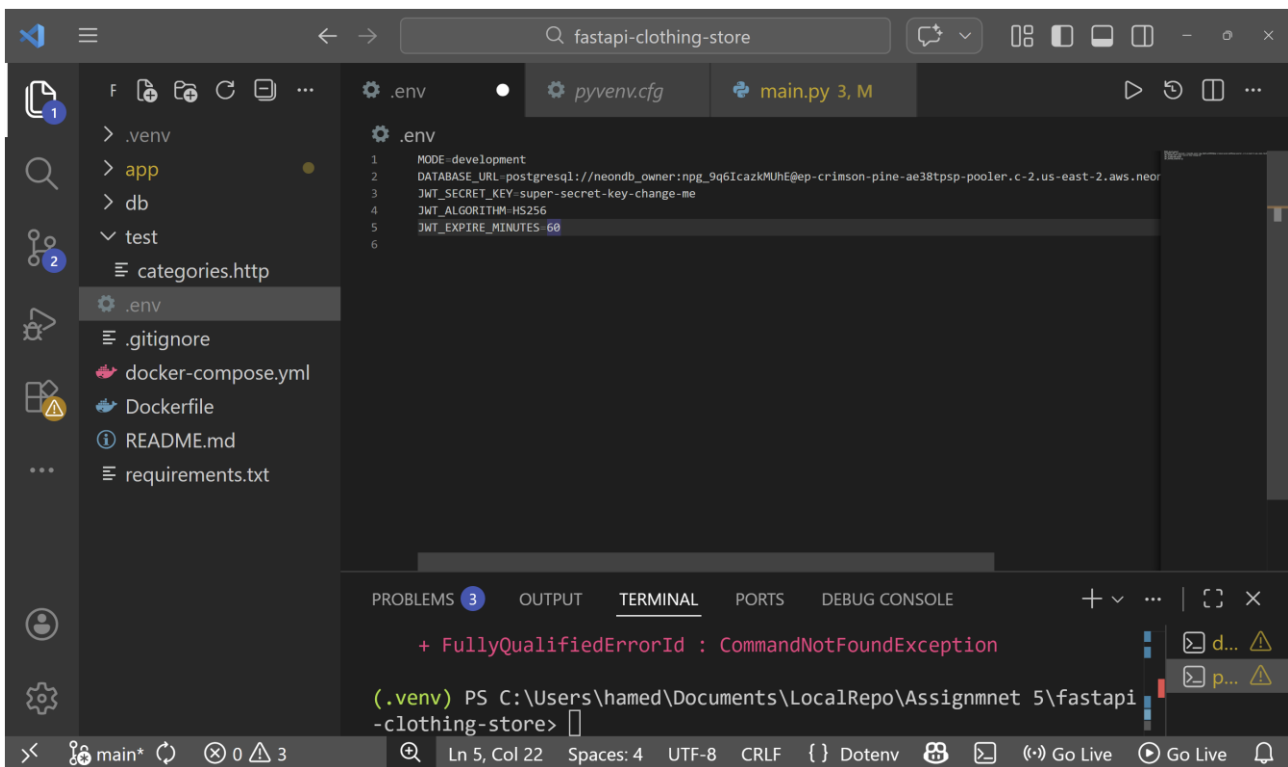
Directory: C:\Users\hamed\Documents\LocalRepo\Assignmnet
5\fastapi-clothing-store\.venv

Mode                LastWriteTime         Length Name
----                -
d-----          2/8/2026 10:27 PM             Include
d-----          2/8/2026 10:27 PM             Lib
d-----          2/8/2026 10:27 PM             Scripts
-a-----          2/8/2026 10:27 PM           503 pyenvn.cfg
```


Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

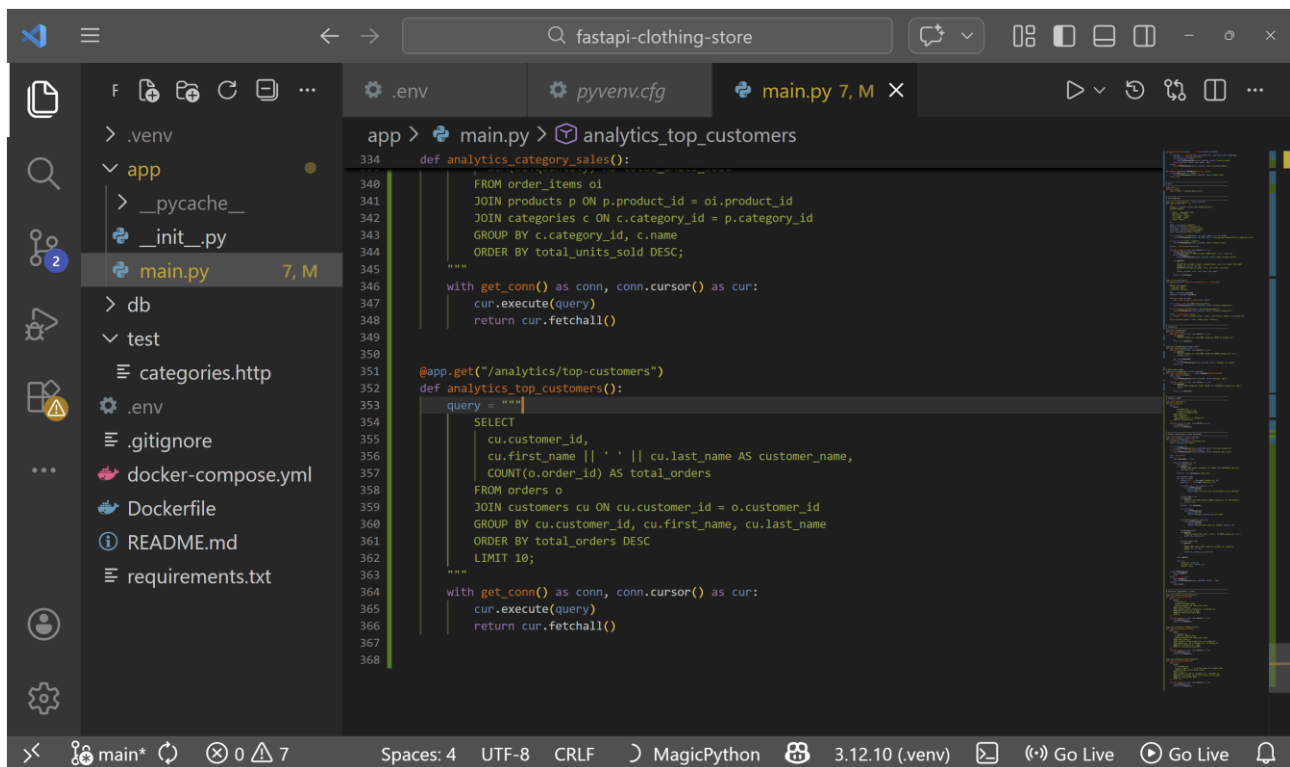
By: Hamed Ahmadinia



Assignment 5: Clothing Store API with Analytics

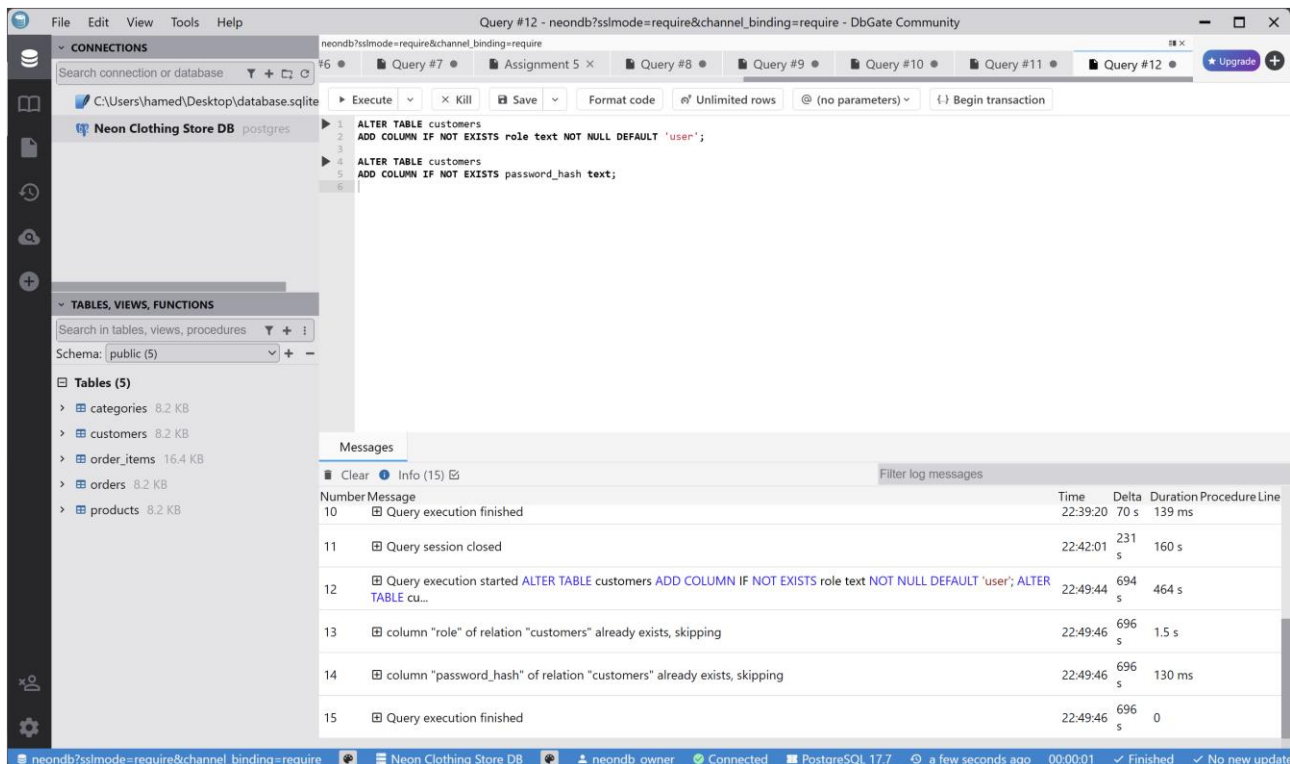
Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



The screenshot shows a VS Code editor with a project named 'fastapi-clothing-store'. The file explorer on the left shows a directory structure with files like `.env`, `pyvenv.cfg`, `main.py`, `__init__.py`, `db`, `test`, `categories.http`, `.gitignore`, `docker-compose.yml`, `Dockerfile`, `README.md`, and `requirements.txt`. The main editor displays the `main.py` file, which contains two FastAPI endpoints for analytics:

```
334 def analytics_category_sales():
335     """
336     FROM order_items oi
337     JOIN products p ON p.product_id = oi.product_id
338     JOIN categories c ON c.category_id = p.category_id
339     GROUP BY c.category_id, c.name
340     ORDER BY total_units_sold DESC;
341     """
342     with get_conn() as conn, conn.cursor() as cur:
343         cur.execute(query)
344         return cur.fetchall()
345
346 @app.get("/analytics/top-customers")
347 def analytics_top_customers():
348     """
349     SELECT
350     cu.customer_id,
351     cu.first_name || ' ' || cu.last_name AS customer_name,
352     COUNT(o.order_id) AS total_orders
353     FROM orders o
354     JOIN customers cu ON cu.customer_id = o.customer_id
355     GROUP BY cu.customer_id, cu.first_name, cu.last_name
356     ORDER BY total_orders DESC
357     LIMIT 10;
358     """
359     with get_conn() as conn, conn.cursor() as cur:
360         cur.execute(query)
361         return cur.fetchall()
362
363
364
365
366
367
368
```



The screenshot shows the DbGate Community interface with a connection to 'Neon Clothing Store DB' (PostgreSQL). The 'Query #12' tab is active, displaying the following SQL queries:

```
1 ALTER TABLE customers
2 ADD COLUMN IF NOT EXISTS role text NOT NULL DEFAULT 'user';
3
4 ALTER TABLE customers
5 ADD COLUMN IF NOT EXISTS password_hash text;
6
```

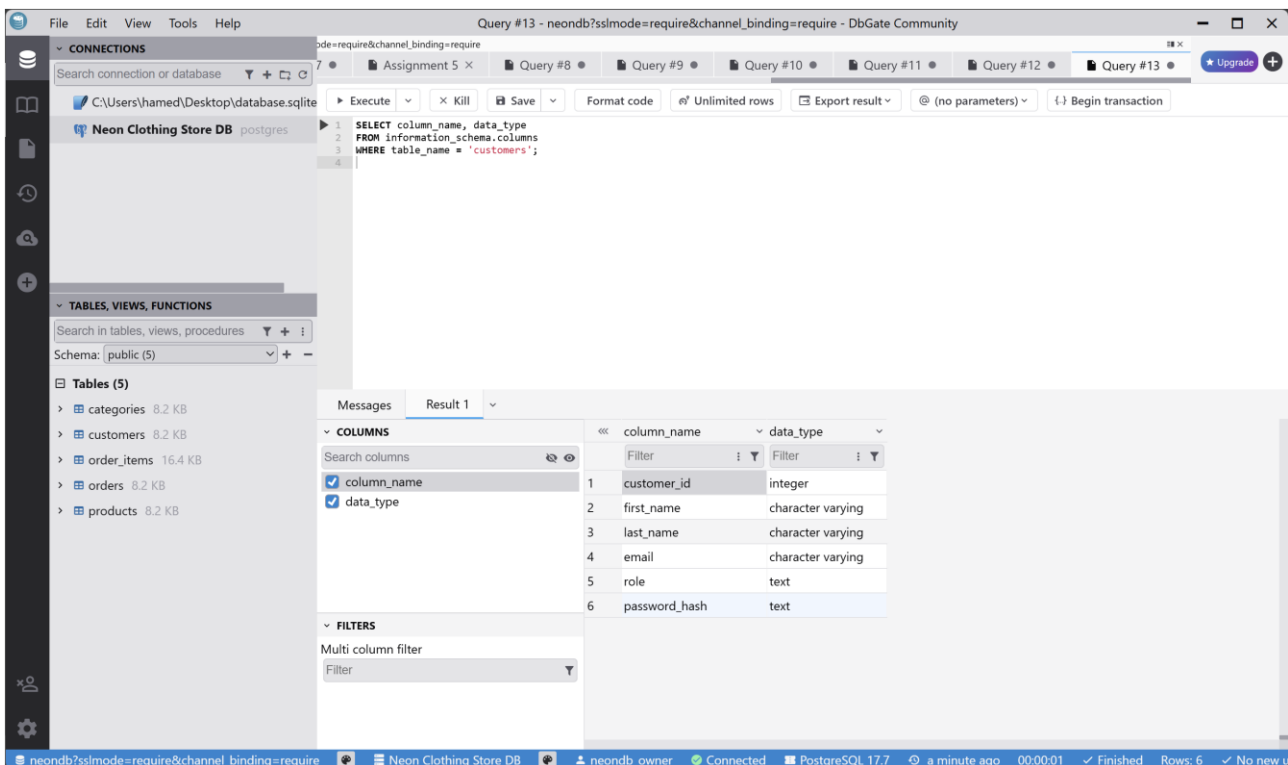
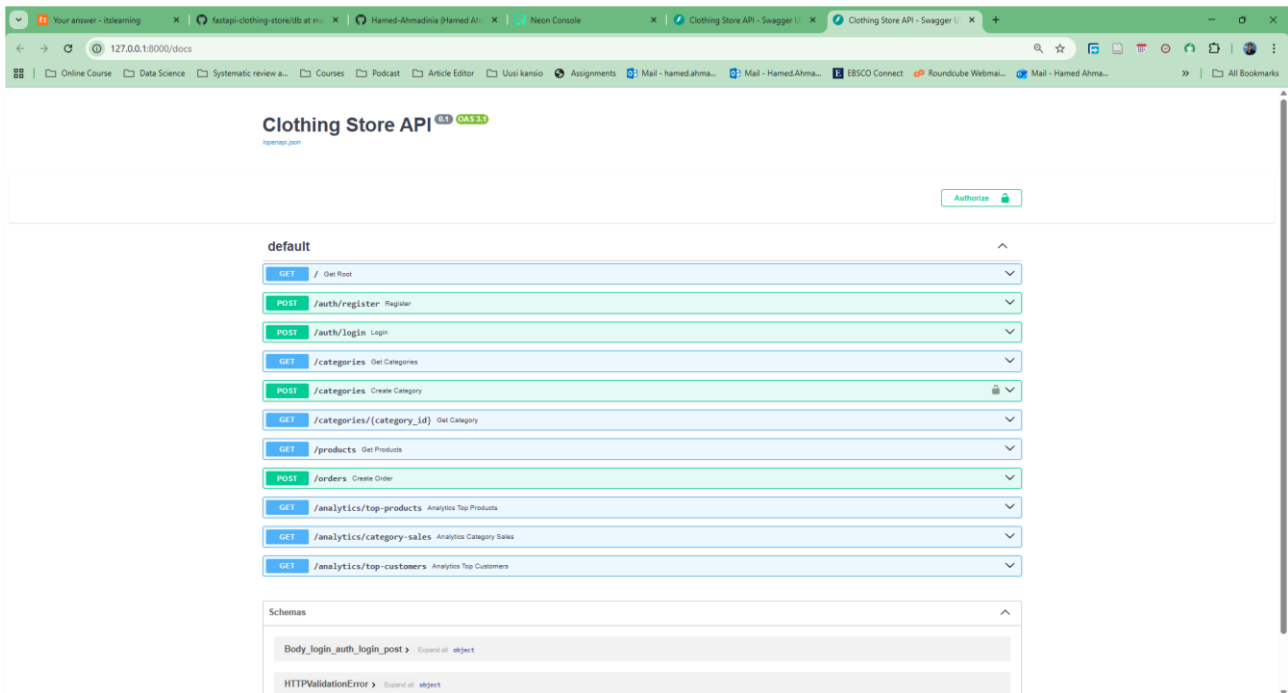
The 'Messages' tab shows the execution results for Query #12:

Number	Message	Time	Delta	Duration	Procedure	Line
10	Query execution finished	22:39:20	70 s	139 ms		
11	Query session closed	22:42:01	231 s	160 s		
12	Query execution started ALTER TABLE customers ADD COLUMN IF NOT EXISTS role text NOT NULL DEFAULT 'user'; ALTER TABLE cu...	22:49:44	694 s	464 s		
13	column "role" of relation "customers" already exists, skipping	22:49:46	696 s	1.5 s		
14	column "password_hash" of relation "customers" already exists, skipping	22:49:46	696 s	130 ms		
15	Query execution finished	22:49:46	696 s	0		

Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

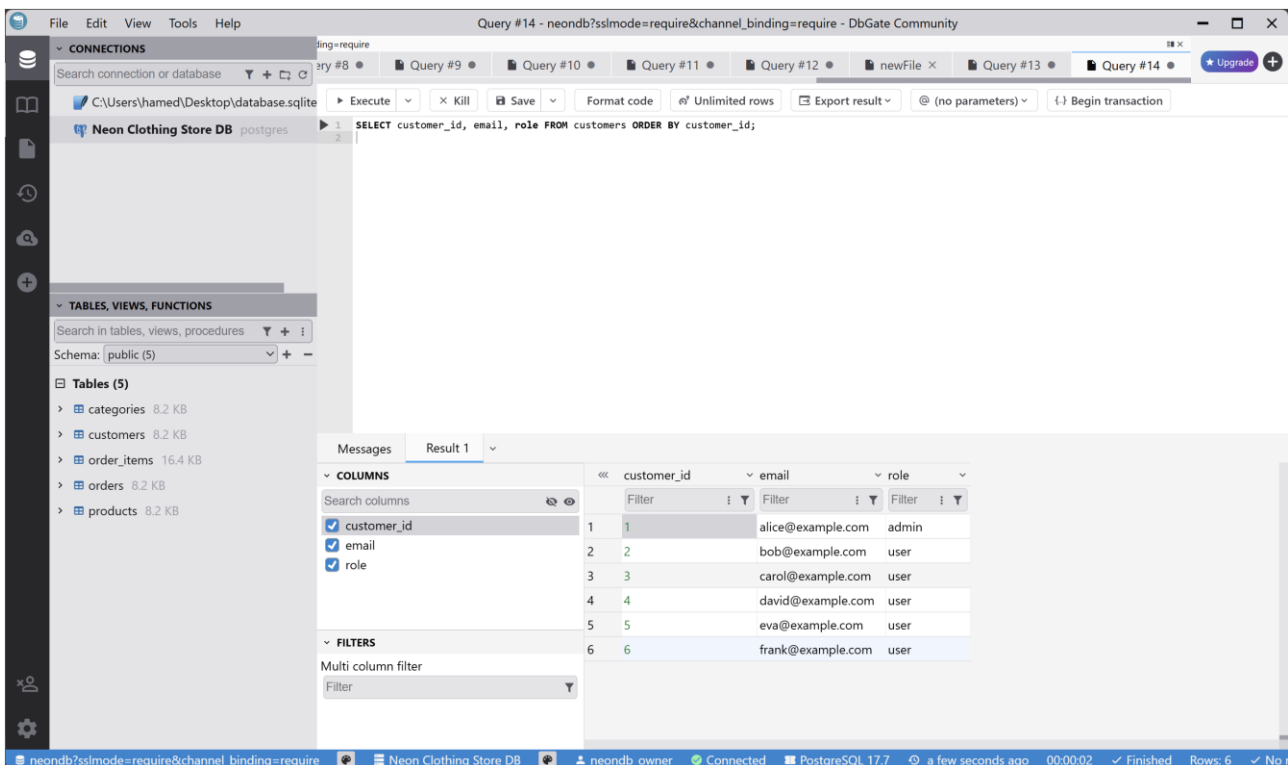
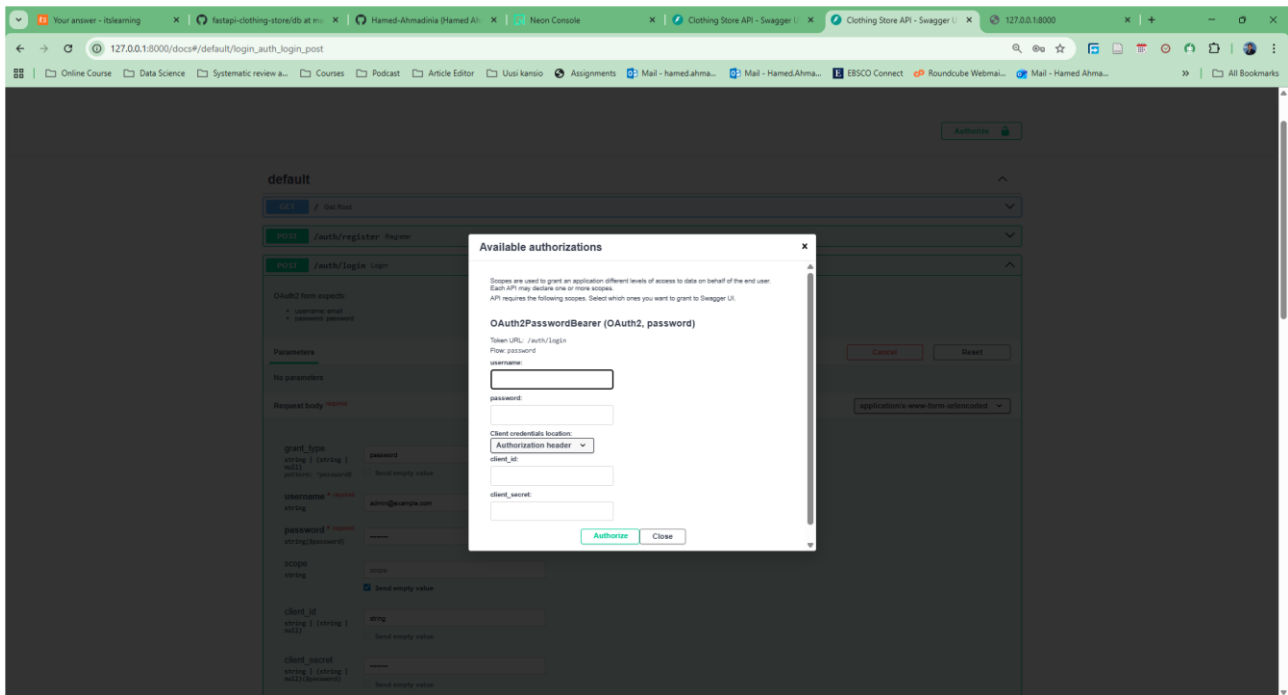
By: Hamed Ahmadinia



Assignment 5: Clothing Store API with Analytics

Course: Cloud Computing and Data Engineering

By: Hamed Ahmadinia



Assignment 5: Clothing Store API with Analytics
Course: Cloud Computing and Data Engineering
By: Hamed Ahmadiania

